Guida al progetto «Harden a C application with ASPIS»

Tutto quello che c'è da sapere per capire, rifare e presentare il progetto

Per Alberto Rotolo e Simone Rota

Linguaggi Formali e Compilatori, A.A. 2026/27, Politecnico di Milano

---

Indice

Come leggere questa guida
Parte I. I concetti
1. Il progetto in cinque minuti
   1.1 Cosa chiede la proposta
   1.2 Cosa è stato fatto
   1.3 Il messaggio principale dei risultati
   1.4 Il repository
   1.5 Una nota sulla dimensione del progetto
2. Compilatori e LLVM: il minimo indispensabile
   2.1 Cosa fa un compilatore
   2.2 LLVM
   2.3 L'IR di LLVM
   2.4 Basic block e grafo di controllo del flusso
   2.5 I pass
   2.6 I puntatori opachi e la deduzione dei tipi
   2.7 Un minimo di x86-64
3. Guasti transitori e tolleranza ai guasti via software
   3.1 Soft error
   3.2 Guasto, errore, fallimento
   3.3 Gli esiti di un guasto
   3.4 Protezione hardware e protezione software
   3.5 Modelli di guasto
   3.6 Fault injection
4. Le tecniche implementate da ASPIS
   4.1 EDDI: duplicare le istruzioni
   4.2 Il controllo del flusso a firme
   4.3 CFCSS
   4.4 RASM
   4.5 Combinare dati e flusso
Parte II. Il lavoro fatto
5. ASPIS in pratica
   5.1 Il repository di ASPIS
   5.2 Compilare ASPIS
   5.3 Usare ASPIS
   5.4 La pipeline di `aspis.sh`

5.5 FuncRetToRef
5.6 Come EDDI trasforma il codice
5.7 Le altre parti
5.8 Come si vede RASM nel codice
6. Le quattro applicazioni
6.1 Perché queste e con quali regole
6.2 `aes128.c`: cifratura AES-128
6.3 `fft.c`: trasformata di Fourier
6.4 `huffman.c`: compressione di Huffman
6.5 `pid_control.c`: controllore PID
7. I test di compatibilità
7.1 Il sistema di test di ASPIS
7.2 Cosa abbiamo fatto
7.3 I risultati
8. I tre bug e le correzioni
8.1 Come si cerca un bug in un pass
8.2 Bug 1: i controlli che leggono da un puntatore nullo
8.3 Bug 2: la copia dopo le chiamate di libreria
8.4 Bug 3: RASM non controllava i blocchi creati da EDDI
9. L'iniettore di guasti (`fi.c`)
9.1 `ptrace` in breve
9.2 `fi profile`: l'esecuzione di riferimento
9.3 Riconoscere un salto condizionale
9.4 `fi inject`: gli esperimenti
9.5 Cosa l'iniettore non fa
10. La campagna e la statistica
10.1 Il file di configurazione
10.2 Cosa fa `campaign.py`
10.3 Percentuali e intervalli di confidenza
10.4 La media geometrica
10.5 Gli script di analisi
11. I risultati, spiegati
11.1 Il costo delle protezioni
11.2 Bit-flip nei registri
11.3 Salti condizionali invertiti
11.4 Il bug 3 nei numeri
11.5 Per applicazione
11.6 Il messaggio in tre frasi
Parte III. Pratica ed esame
12. L'integrazione in ASPIS
13. Rifare tutto da zero
14. Mappa dei file
15. Cosa è stato tolto (la versione completa)
16. Domande probabili all'orale
17. Glossario

Come leggere questa guida

Questa guida spiega **tutto** il progetto, partendo da zero. Si assume solo quello che uno studente di ingegneria informatica conosce già: il linguaggio C, le basi di architettura dei calcolatori (registri, istruzioni, memoria) e di sistemi operativi (processi, segnali, `fork` / `exec`). Non si assume nessuna conoscenza del corso di Linguaggi Formali e Compilatori, di LLVM o di tolleranza ai guasti.

La guida è divisa in tre parti:

- **Parte I, i concetti** (capitoli 1-4): cosa chiede il progetto, cos'è un compilatore e com'è fatto LLVM, cosa sono i guasti transitori e come si proteggono i programmi via software, quali tecniche implementa ASPIS.
- **Parte II, il lavoro fatto** (capitoli 5-11): come si usa ASPIS, le quattro applicazioni, i test, i tre bug trovati e corretti, l'iniettore di guasti, la campagna di fault injection e i suoi risultati.
- **Parte III, pratica ed esame** (capitoli 12-17): come integrare il lavoro in ASPIS, come rifare tutti gli esperimenti, la mappa dei file, cosa è stato tolto dalla prima versione, le domande probabili all'orale con le risposte e un glossario.

Il consiglio è leggere i capitoli nell'ordine, con il repository aperto accanto: ogni volta che si parla di un file, il percorso è scritto in `questo formato`, relativo alla cartella `ASPIS-Project/` del repository (salvo dove indicato diversamente).

Il report ufficiale del progetto, quello che leggeranno i supervisori, è `ASPIS-Project/report/report.pdf` ed è in inglese. Questa guida ne segue la struttura, ma spiega molto di più: il perché delle scelte, i meccanismi interni e i dettagli del codice. Dove serve, riporta i termini inglesi usati nel report.

Parte I. I concetti

1. Il progetto in cinque minuti

1.1 Cosa chiede la proposta

Il progetto «Harden a C application with ASPIS» (supervisori Davide Baroffio ed Emilio Corigliano, del gruppo HEAPLab) ha tre obiettivi:

1. **proteggere applicazioni C con ASPIS**, un insieme di trasformazioni del compilatore LLVM che rendono un programma capace di accorgersi dei guasti hardware;
2. **verificare la protezione con la fault injection**, cioè iniettando guasti artificiali nel programma in esecuzione e osservando cosa succede;
3. **integrare le applicazioni nel branch principale di ASPIS** come test di regressione, così che ogni modifica futura di ASPIS venga provata anche su di esse.

I supervisori hanno anche scritto (file `Buongiorno,.txt` nella radice del repository) che si può usare l'intelligenza artificiale, anche per tutto il progetto, a patto di dichiararlo nel report, e che ci sarà una chiamata in cui si discuterà il lavoro usando il report come riferimento.

1.2 Cosa è stato fatto

- **Quattro applicazioni** scritte apposta, piccole e capaci di controllare da sole il proprio risultato: cifratura AES-128, trasformata di Fourier (FFT), compressione di Huffman, controllo PID della velocità di un motore.
- **Test di compatibilità** con il sistema di test ufficiale di ASPIS, che compila ogni programma con 55 combinazioni di tecniche. Sul branch principale di ASPIS funzionano solo 100 delle 220 combinazioni delle nostre applicazioni: con EDDI, sEDDI e FDSC (le tecniche che duplicano tutto il programma) le quattro applicazioni vanno in crash.
- **Tre bug di ASPIS trovati e corretti**. Due sono in EDDI e causavano i crash; il terzo è in RASM ed è stato scoperto grazie alla fault injection. Con le correzioni tutte le 220 combinazioni funzionano e i 48 test che ASPIS aveva già danno gli stessi risultati di prima.
- **Un iniettore di guasti** scritto in C (`fault_injection/injector/fi.c`) che, usando la chiamata di sistema `ptrace` (quella dei debugger), ferma il programma in un istante casuale e inverte un bit di un registro oppure fa andare un salto condizionale dalla parte sbagliata.
- **Una campagna di fault injection** di 56.000 esperimenti: 4 applicazioni, 6 configurazioni di protezione, 2 modelli di guasto, 1000 guasti per combinazione, più 8000 esperimenti per misurare l'effetto del terzo bug.
- **Quattro patch** pronte per il repository ufficiale di ASPIS (le tre correzioni e l'aggiunta dei test), non ancora inviate.
- **Il report** in inglese, con la dichiarazione sull'uso dell'AI.

1.3 Il messaggio principale dei risultati

- **EDDI** (duplicazione dei dati) riduce drasticamente i risultati sbagliati non rilevati causati da bit-flip nei registri, ma raddoppia abbondantemente il costo: il programma esegue circa 3,8 volte le istruzioni.

- EDDI **non vede** un salto condizionale andato dalla parte sbagliata, perché le due copie del calcolo seguono lo stesso salto. Lo vedono invece le tecniche di **controllo del flusso a firme**, ma non tutte: **RASM** rileva quasi tutti i salti invertiti, **CFCSS** no, perché verifica solo che il salto vada in un blocco "ammesso", non in quello giusto.
- Combinare EDDI con RASM avrebbe dovuto dare il meglio dei due mondi, ma la fault injection ha mostrato che la combinazione rilevava i salti invertiti **peggio** di RASM da solo: era il terzo bug, ora corretto.

I numeri precisi sono nel capitolo 11.

1.4 Il repository

Cartella o file	Contenuto
`ASPIS-Project/benchmarks/`	le quattro applicazioni
`ASPIS-Project/aspis-integration/`	le quattro patch per ASPIS, le voci dei test, due programmi minimi che riproducono i bug
`ASPIS-Project/fault_injection/`	l'iniettore (`injector/fi.c`), la campagna (`campaign.py`, `campaign.toml`), l'analisi (`analyze.py`, `export.py`)
`ASPIS-Project/results/`	i log dei test e i risultati della campagna
`ASPIS-Project/report/`	il report (`report.md`, `report.pdf`) e lo script che lo genera
`guida/`	questa guida (fuori dalla cartella del progetto)

1.5 Una nota sulla dimensione del progetto

La prima versione del progetto era molto più grande: 22 programmi di test, 23 bug corretti in ASPIS, un iniettore più sofisticato e una campagna di 240.900 esperimenti. Era troppo per un progetto che vale il 30% di un esame da 5 CFU, quindi è stata ridotta a una dimensione realistica per due studenti. La versione completa non è andata persa: è nella storia di git, al commit `e1d7e8b`. Il capitolo 15 riassume cosa conteneva.

2. Compilatori e LLVM: il minimo indispensabile

2.1 Cosa fa un compilatore

Un compilatore traduce un programma da un linguaggio sorgente (per noi il C) a codice macchina. Lo fa in fasi successive:

1. **analisi lessicale**: il testo viene spezzato in *token* (parole chiave, identificatori, numeri, operatori). È qui che entrano in gioco le espressioni regolari e gli automi a stati finiti, la parte "linguaggi formali" del corso;
2. **analisi sintattica** (*parsing*): i token vengono organizzati in un albero sintattico secondo una grammatica libera dal contesto;
3. **analisi semantica**: si controllano tipi, dichiarazioni, regole del linguaggio;
4. **generazione della rappresentazione intermedia** (*IR, intermediate representation*): il programma viene tradotto in un linguaggio più semplice e regolare, indipendente sia dal C sia dalla macchina;
5. **ottimizzazione**: trasformazioni dell'IR che lo rendono più veloce o più piccolo senza cambiarne il significato;
6. **generazione del codice**: l'IR viene tradotto nelle istruzioni del processore (per noi x86-64), con l'allocazione dei registri.

Le fasi 1-4 formano il **front-end** (dipende dal linguaggio sorgente), la 5 il **middle-end** (non dipende né dal sorgente né dalla macchina), la 6 il **back-end** (dipende dalla macchina). Il vantaggio di un IR comune è che si scrive una trasformazione una volta sola e funziona per tutti i linguaggi e tutte le architetture.

ASPIS lavora nel middle-end: le sue protezioni sono **trasformazioni dell'IR**. Non tocca né il parsing del C né la generazione del codice x86.

2.2 LLVM

LLVM è l'infrastruttura di compilazione più usata nella ricerca e in molti prodotti (Apple, Google, Rust, Swift usano LLVM). I programmi che ci interessano sono:

Programma	Cosa fa
`clang`	front-end per C e C++: produce l'IR (con `-S -emit-llvm`) oppure direttamente l'eseguibile
`opt`	esegue dei *pass* (trasformazioni o analisi) su un file IR e scrive l'IR trasformato
`llvm-link`	unisce più file IR in un unico modulo
`clang` su un file `.ll`	fa da back-end: traduce l'IR in codice macchina e collega l'eseguibile
`llvm-size`	stampa la dimensione delle sezioni di un eseguibile (`.text` è il codice)

Il file IR in forma testuale ha estensione `.ll`. Si può leggere, ed è molto utile farlo: quando ASPIS ha un problema, il primo passo è guardare l'IR che produce.

Noi usiamo **LLVM 21.1.8**, la versione richiesta da ASPIS, scaricata già compilata dal sito di LLVM.

2.3 L'IR di LLVM

Un file IR contiene un **modulo**, cioè un insieme di variabili globali e di funzioni. Ogni funzione è fatta di **basic block** (blocchi di base), ciascuno con un'etichetta, e ogni blocco è una sequenza di **istruzioni**.

L'IR è in forma **SSA** (*static single assignment*): ogni valore ha un nome (`%a`, `%0`, `%add` …) e viene assegnato **una sola volta**. Le variabili del C che cambiano valore non diventano valori SSA, ma celle di memoria sullo stack, create

con `alloca` e lette e scritte con `load` e `store`. Questo è esattamente quello che fa `clang` senza ottimizzazioni (`-O0`), che è il modo in cui lo usa ASPIS.

Esempio. Questa funzione C:

```c
int sum(int *a, int n) {
  int s = 0;
  for (int i = 0; i < n; i++)
    s += a[i];
  return s;
}
```

diventa (con `clang -S -emit-llvm -O0`, tolti alcuni attributi):

```
define i32 @sum(ptr %a, i32 %n) {
entry:
  %a.addr = alloca ptr               ; cella sullo stack per il parametro a
  %n.addr = alloca i32               ; ... per n
  %s = alloca i32                    ; variabile locale s
  %i = alloca i32                    ; variabile locale i
  store ptr %a, ptr %a.addr          ; copia i parametri nelle celle
  store i32 %n, ptr %n.addr
  store i32 0, ptr %s                ; s = 0
  store i32 0, ptr %i                ; i = 0
  br label %for.cond                 ; salto incondizionato

for.cond:                            ; condizione del ciclo
  %0 = load i32, ptr %i
  %1 = load i32, ptr %n.addr
  %cmp = icmp slt i32 %0, %1         ; i < n ? (slt = signed less than)
  br i1 %cmp, label %for.body, label %for.end   ; salto condizionale

for.body:                            ; s += a[i]
  %2 = load ptr, ptr %a.addr
  %3 = load i32, ptr %i
  %idxprom = sext i32 %3 to i64      ; estensione di segno a 64 bit
  %arrayidx = getelementptr inbounds i32, ptr %2, i64 %idxprom   ; &a[i]
  %4 = load i32, ptr %arrayidx       ; a[i]
  %5 = load i32, ptr %s
  %add = add nsw i32 %5, %4
  store i32 %add, ptr %s
  br label %for.inc

for.inc:                             ; i++
  %6 = load i32, ptr %i
  %inc = add nsw i32 %6, 1
  store i32 %inc, ptr %i
  br label %for.cond

for.end:
  %7 = load i32, ptr %s
  ret i32 %7                         ; return s
}
```

Le istruzioni da conoscere:

Istruzione	Significato
`alloca T`	riserva sullo stack lo spazio per un valore di tipo `T`, restituisce il puntatore
`load T, ptr p`	legge un valore di tipo `T` dall'indirizzo `p`
`store T v, ptr p`	scrive `v` all'indirizzo `p`
`getelementptr` (GEP)	calcola un indirizzo (elemento di un array, campo di una struct) senza leggere la memoria
`add`, `sub`, `mul`, `xor`, …	aritmetica intera; `fadd`, `fmul`, … per i numeri in virgola mobile
`icmp`, `fcmp`	confronti (interi e in virgola mobile): danno un `i1`, cioè un booleano
`br i1 c, label A, label B`	salto condizionale: va in `A` se `c` è vero, altrimenti in `B`
`br label A`	salto incondizionato
`select i1 c, T x, T y`	`c ? x : y` senza salti
`call`	chiamata di funzione
`ret`	ritorno dalla funzione
`phi`	sceglie un valore a seconda del blocco da cui si è arrivati (con `-O0` è rara)

I tipi: `i1`, `i8`, `i32`, `i64` sono interi di 1, 8, 32, 64 bit; `float`, `double`; `ptr` è un puntatore; poi ci sono struct (`{ i32, float }`) e array (`[16 x i8]`). I nomi che cominciano con `@` sono globali (funzioni e variabili globali), quelli con `%` sono locali alla funzione.

ASPIS non usa i nomi leggibili come `%a.addr`: `clang` di default li scarta e numera i valori (`%0`, `%1`, …). Il significato non cambia.

2.4 Basic block e grafo di controllo del flusso

Un **basic block** è una sequenza di istruzioni senza salti al suo interno: si entra solo dall'inizio e si esce solo dalla fine. L'ultima istruzione è un *terminatore* (`br`, `ret`, `unreachable`, …), che dice dove si va dopo.

I blocchi di una funzione e i salti tra di loro formano il **grafo di controllo del flusso** (CFG, *control-flow graph*). Per `sum`:



Un'esecuzione corretta percorre sempre archi di questo grafo. Le tecniche di *control-flow checking* (capitolo 4) controllano proprio questo: che il programma segua gli archi giusti.

2.5 I pass

Un **pass** è un pezzo di codice che analizza o trasforma l'IR: per esempio `simplifycfg` semplifica il grafo dei blocchi, `lower-switch` trasforma gli `switch` in catene di `br`. Si eseguono con `opt`:

```
opt --passes="lower-switch" in.ll -o out.ll
```

Un pass può stare fuori dal codice di LLVM, in una libreria condivisa (un *plugin*) caricata al momento:

```
opt -load-pass-plugin=build/passes/libEDDI.so --passes="eddi-verify" in.ll -o out.ll
```

Le protezioni di ASPIS sono pass di questo tipo (*out-of-tree*): sono scritte in C++ usando le classi di LLVM (`Module`, `Function`, `BasicBlock`, `Instruction`, `IRBuilder` per creare nuove istruzioni) e compilate in file `.so`.

2.6 I puntatori opachi e la deduzione dei tipi

Nelle versioni recenti di LLVM i puntatori sono **opachi**: il tipo `ptr` non dice a cosa punta. Una `load i32, ptr %p` dice che in quel punto si legge un `i32`, ma `%p` da solo non lo dice. Questo crea un problema ad ASPIS: per confrontare due puntatori deve confrontare i valori puntati, e quindi deve sapere di che tipo sono. Per questo ASPIS usa una libreria esterna, **TypeDeductionAnalysis** (TDA), che ricostruisce il tipo puntato guardando come il puntatore viene usato.

2.7 Un minimo di x86-64

L'iniettore di guasti lavora sul programma già compilato, quindi serve qualche nozione di x86-64:

- **16 registri generali** a 64 bit: `rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp`, `rsp`, `r8` … `r15`. `rsp` è il puntatore alla cima dello stack, `rbp` di solito punta al record di attivazione (*frame*) della funzione corrente;
- `rip`, il *program counter*: l'indirizzo della prossima istruzione;
- `rflags`, il registro dei flag, impostato dalle istruzioni aritmetiche e di confronto: `ZF` (risultato zero), `SF` (segno), `CF` (riporto), `OF` (overflow), `PF` (parità);
- i **salti condizionali** (`jcc`: `je`, `jne`, `jl`, `jge`, …) saltano o meno a seconda dei flag. Sono codificati come `0x70`-`0x7f` seguito da uno spostamento di 8 bit, oppure `0x0f 0x80`-`0x8f` seguito da uno spostamento di 32 bit. I 4 bit bassi dell'opcode sono il *condition code* (`cc`);
- con `-O0` ogni variabile vive sullo stack, a un indirizzo come `[rbp - 20]`.

Il ciclo di `sum` compilato con `-O0` (sintassi Intel):

```
.LBB0_1:                                ; for.cond
    mov  eax, dword ptr [rbp - 20]      ; eax = i
    cmp  eax, dword ptr [rbp - 12]      ; confronta i con n (imposta i flag)
    jge  .LBB0_4                        ; se i >= n esce dal ciclo
    mov  rax, qword ptr [rbp - 8]       ; rax = a
    movsxd rcx, dword ptr [rbp - 20]    ; rcx = i (esteso a 64 bit)
    mov  eax, dword ptr [rax + 4*rcx]   ; eax = a[i]
    add  eax, dword ptr [rbp - 16]      ; eax += s
    mov  dword ptr [rbp - 16], eax      ; s = eax
    ...                                 ; i++
    jmp  .LBB0_1
.LBB0_4:
```

Un bit-flip in `rcx` subito prima di `mov eax, dword ptr [rax + 4*rcx]` fa leggere un elemento sbagliato (o un indirizzo non valido, e allora il programma va in crash). Invertire il `jge` fa uscire dal ciclo troppo presto o eseguire un'iterazione di troppo. Sono esattamente i due modelli di guasto del progetto.

3. Guasti transitori e tolleranza ai guasti via software

3.1 Soft error

Una particella (un neutrone dei raggi cosmici, una particella alfa emessa dai materiali del chip) che colpisce un transistor può depositare abbastanza carica da cambiare il valore di un bit: un registro, una cella di memoria, un latch della pipeline. Il componente non si rompe: al prossimo accesso funziona di nuovo. Si parla di **guasto transitorio**, *soft error* o *SEU* (*single event upset*).

Il problema è serio nello spazio e in alta quota (avionica, satelliti) ma anche a terra nei sistemi critici (automotive, ferroviario, medicale), e cresce con la miniaturizzazione: transistor più piccoli e tensioni più basse significano che basta meno carica per invertire un bit.

3.2 Guasto, errore, fallimento

La terminologia classica distingue tre livelli:

- **guasto** (*fault*): la causa fisica, per esempio il bit invertito;
- **errore** (*error*): lo stato del sistema diventato sbagliato, per esempio un valore errato in un registro che verrà usato;
- **fallimento** (*failure*): il servizio fornito è sbagliato, per esempio il programma stampa un risultato errato.

Non tutti i guasti diventano errori (il registro poteva non contenere nulla di utile) e non tutti gli errori diventano fallimenti.

3.3 Gli esiti di un guasto

Quando si inietta un guasto in un programma, l'esecuzione può finire in cinque modi, che ritroveremo in tutti i risultati:

Esito	Significato
masked (mascherato)	il programma termina normalmente con l'output corretto: il guasto non ha avuto effetti (registro non usato, valore sovrascritto, differenza irrilevante)
detected (rilevato)	un controllo inserito dalla protezione si è accorto del guasto e ha chiamato il gestore d'errore
crash	il programma è stato ucciso da un segnale, tipicamente `SIGSEGV` (accesso a un indirizzo non valido)
hang (blocco)	il programma non termina (ciclo infinito) entro un tempo limite
SDC (*silent data corruption*)	il programma termina "normalmente" ma con un risultato sbagliato

L'esito peggiore è la **SDC**: nessuno se ne accorge, e un sistema che si fida del risultato agisce su un dato sbagliato. Crash, hang e rilevamenti sono invece errori *rilevati* (*DUE, detected unrecoverable errors*): il sistema sa che qualcosa è andato storto e può reagire (riavviare, ripetere il calcolo, passare a uno stato sicuro).

Lo scopo delle tecniche che studiamo è quindi **trasformare le SDC in rilevamenti**. Non correggono l'errore: lo segnalano.

3.4 Protezione hardware e protezione software

Ci si può proteggere in hardware: memorie con codici di correzione (ECC), triplicazione dei circuiti con votazione (TMR), processori che eseguono lo stesso programma in parallelo e confrontano i risultati (*lockstep*). Funziona bene, ma costa: chip speciali, più area, più consumo.

L'alternativa è la **SIHFT** (*Software-Implemented Hardware Fault Tolerance*): si trasforma il programma in modo che si accorga da solo dei guasti, e lo si esegue su hardware normale ed economico. Il prezzo si paga in **tempo di esecuzione** (il programma fa più lavoro) e in **dimensione del codice**.

Le tecniche SIHFT si dividono in due famiglie:

- **protezione del flusso dei dati** (*data-flow*): si duplica il calcolo e si confrontano le due copie. Se un guasto altera una sola copia, il confronto lo rivela;
- **controllo del flusso di esecuzione** (*control-flow checking*, CFC): a ogni basic block si associa una firma e durante l'esecuzione si verifica che i blocchi vengano eseguiti in un ordine permesso dal grafo di controllo del flusso. Serve contro i guasti che fanno saltare il programma nel posto sbagliato.

Le due famiglie sono complementari, e si possono combinare.

3.5 Modelli di guasto

Per valutare una tecnica bisogna decidere quali guasti simulare. Un **modello di guasto** descrive il guasto in modo astratto. I più usati:

- **bit-flip in un registro**: un bit di un registro del processore viene invertito in un istante qualsiasi. Rappresenta i guasti nel banco dei registri e, approssimativamente, quelli nelle unità di calcolo;
- **salto condizionale invertito**: un salto condizionale va dalla parte opposta a quella decisa dalla condizione. Rappresenta un guasto nei flag o nell'unità che gestisce i salti. Il programma resta su archi "permessi" del grafo, ma sbaglia strada;
- **salto a un indirizzo sbagliato**: il program counter viene alterato e il programma continua da un'istruzione qualsiasi;
- **bit-flip in memoria**: un bit di una cella di memoria viene invertito.

Nel progetto usiamo i primi due (capitolo 9).

3.6 Fault injection

Per misurare quanto è efficace una protezione si fa **fault injection**: si esegue il programma molte volte, ogni volta con un guasto artificiale in un punto scelto a caso, e si contano gli esiti. Il guasto si può iniettare simulando l'hardware, con apparecchiature fisiche (fasci di particelle, laser), oppure via software, modificando lo stato del programma mentre è in esecuzione. Noi usiamo l'ultima strada: un programma che, come un debugger, ferma il programma sotto test e ne modifica i registri.

Siccome i guasti sono scelti a caso, ogni percentuale misurata è una **stima**, e va accompagnata da un **intervallo di confidenza**: con 4000 guasti una percentuale dello 0,1% significa 4 casi, e la vera probabilità potrebbe essere un po' più alta o più bassa. Il capitolo 10 spiega come si calcola l'intervallo.

4. Le tecniche implementate da ASPIS

4.1 EDDI: duplicare le istruzioni

EDDI (*Error Detection by Duplicated Instructions*, Oh, Shirvani e McCluskey, 2002) è la tecnica di base della protezione dei dati. L'idea: ogni istruzione viene eseguita due volte, su due copie separate dei dati, e prima dei punti "pericolosi" si controlla che le due copie coincidano.

In pseudo-C, un'istruzione `a = b + c; x[i] = a;` diventa:

```
a  = b  + c;          // copia originale
a2 = b2 + c2;         // copia duplicata, su variabili separate
if (a != a2 || i != i2) DataCorruption_Handler();   // controllo
x[i]  = a;            // la store originale
x2[i2] = a2;          // e la sua copia, su un array duplicato
```

Ogni variabile ha la sua copia (`b2`, `c2`, `x2`...): locali, globali, e anche la memoria a cui puntano i puntatori, quando è possibile. Un guasto che altera un valore di una sola copia viene scoperto al primo controllo che coinvolge quel valore.

Dove si controlla. Controllare dopo ogni istruzione costerebbe troppo. EDDI controlla nei **punti di sincronizzazione**, cioè dove un valore sbagliato "esce" dal calcolo duplicato e diventa difficile da recuperare:

- prima di una **store**: un valore sbagliato scritto in memoria verrebbe letto da altre parti del programma (si confrontano il valore e l'indirizzo);
- prima di un **salto condizionale**: una condizione sbagliata manderebbe il programma sulla strada sbagliata (si confronta la condizione);
- prima di una **chiamata**: gli argomenti sbagliati verrebbero passati alla funzione chiamata.

Costo. Il programma esegue circa il doppio delle istruzioni, più i confronti e i salti dei controlli: nei nostri programmi EDDI esegue circa 3,8 volte le istruzioni dell'originale (con `-O0`, dove ogni valore passa dallo stack, il costo relativo è più alto che con il codice ottimizzato).

Cosa non vede. Se un guasto colpisce un valore *prima* che venga duplicato (per esempio un dato letto da un file) o colpisce entrambe le copie allo stesso modo, i confronti non lo vedono. Soprattutto, EDDI esegue **un solo salto** per ogni salto del programma: le due copie della condizione vengono confrontate, poi si salta usando la prima. Se è il salto stesso a sbagliare direzione (guasto nei flag o nell'unità di salto), le due copie dei dati seguono insieme la strada sbagliata e restano coerenti tra loro: EDDI non se ne accorge.

Le varianti in ASPIS. Tutte sono implementate dallo stesso file (`passes/EDDI.cpp`) con interruttori diversi:

Opzione	Tecnica	Differenza rispetto a EDDI
`--eddi`	EDDI	duplica tutto, controlla prima di store, salti e chiamate
`--seddi`	sEDDI (*selective EDDI*)	non controlla prima delle store: costa meno, rileva più tardi
`--fdsc`	FDSC (*Full Duplication with Selective Checking*)	duplica tutto ma controlla solo in un sottoinsieme dei punti
`--reddi`	REDDI	protegge solo le funzioni e le globali annotate con `to_harden`
`--multiple-errbb`		un blocco di errore separato per ogni controllo (utile per capire quale controllo è scattato)
`--coarse-grained`	REPAIR	raggruppa le istruzioni duplicate dopo quelle originali, per allontanarle nel tempo

4.2 Il controllo del flusso a firme

Le tecniche di **control-flow checking** proteggono dai guasti che fanno eseguire i blocchi nell'ordine sbagliato. L'idea comune: ogni basic block ha una **firma** (un numero scelto dal compilatore), e il programma tiene in una variabile la **firma di esecuzione**, aggiornata a ogni passaggio da un blocco all'altro. All'ingresso di ogni blocco si controlla che la firma di esecuzione sia quella del blocco: se il programma è arrivato lì da un punto non previsto, la firma non torna e viene chiamato `SigMismatch_Handler`.

Gli errori di flusso si possono classificare così:

1. salto verso un blocco che **non** è un successore del blocco corrente (arco inesistente nel grafo);
2. salto nel **mezzo** di un blocco;
3. salto verso il successore **sbagliato** tra quelli ammessi: è il caso del salto condizionale invertito, dove l'arco esiste ma non è quello scelto dalla condizione.

Le tecniche differiscono in quali di questi errori rilevano.

4.3 CFCSS

CFCSS (*Control-Flow Checking by Software Signatures*, Oh, Shirvani e McCluskey, 2002). Ogni blocco `i` ha una firma statica `s_i`, diversa per ogni blocco. Per ogni blocco con un solo predecessore `p` il compilatore calcola la differenza `d_i = s_p XOR s_i`. La firma di esecuzione `G` viene aggiornata all'ingresso di ogni blocco e confrontata:

```
G = G ^ d_i;                      // all'ingresso del blocco i
if (G != s_i) SigMismatch_Handler();
```

Esempio con firme di 4 bit: il blocco 1 ha due successori, 2 e 3.

```
s1 = 0001     s2 = 0010     s3 = 0100
d2 = s1 XOR s2 = 0011
d3 = s1 XOR s3 = 0101

esecuzione 1 -> 2:   G = 0001;  all'ingresso di 2: G = 0001 XOR 0011 = 0010 = s2   OK
```

Se un guasto fa saltare dal blocco 1 a un blocco 4 che non è suo successore (per esempio un blocco il cui predecessore è 2, con `d4 = s2 XOR s4`), all'ingresso di 4 si ottiene `G = s1 XOR s2 XOR s4`, diverso da `s4`: **rilevato** (errore di tipo 1).

Ma se il salto condizionale del blocco 1 viene **invertito** e si va in 3 invece che in 2:

```
esecuzione 1 -> 3 (sbagliata):  G = 0001 XOR 0101 = 0100 = s3   nessun errore!
```

L'arco 1 -> 3 esiste, quindi CFCSS lo considera lecito: **non rilevato** (errore di tipo 3). CFCSS controlla che il programma segua *un* arco del grafo, non che segua *quello giusto*.

Per i blocchi con più predecessori serve un accorgimento, la firma di aggiustamento `D`: il blocco predecessore imposta `D` prima di saltare, e il blocco di destinazione calcola `G = G XOR d_i XOR D`, in modo che il risultato sia `s_i` da qualunque predecessore lecito si arrivi.

4.4 RASM

RASM (*Random Additive Signature Monitoring*, Vankeirsbilck e altri, 2017) usa firme casuali e operazioni di somma e sottrazione, e soprattutto aggiorna la firma **prima di uscire** da un blocco in base al successore scelto. Ogni blocco `B` ha due numeri casuali: la firma `sig(B)` e un correttivo `sub(B)`. Nel codice di ASPIS (`passes/RASM.cpp`):

```
// all'ingresso del blocco B
S = S - sub(B);
if (S != sig(B)) SigMismatch_Handler();
...  // corpo del blocco
// alla fine di B, prima del salto condizionale "if (c) goto T; else goto F;"
adj_T = sig(B) - (sig(T) + sub(T));      // costanti calcolate dal compilatore
adj_F = sig(B) - (sig(F) + sub(F));
S = S - (c ? adj_T : adj_F);              // una select, non un salto
if (c) goto T; else goto F;
```

Dopo l'aggiornamento `S` vale `sig(T) + sub(T)` se `c` è vero, `sig(F) + sub(F)` se è falso: proprio il valore che il blocco di destinazione si aspetta. Esempio numerico:

```
sig(B) = 100;   sig(T) = 37, sub(T) = 5;   sig(F) = 58, sub(F) = 9
adj_T = 100 - 42 = 58          adj_F = 100 - 67 = 33

c vero, si va in T (corretto):   S = 100 - 58 = 42;  in T: 42 - 5 = 37 = sig(T)   OK
c vero, ma il salto va in F:     S = 42;              in F: 42 - 9 = 33 != 58     RILEVATO
```

Il punto chiave è che la firma viene aggiornata usando **la stessa condizione** del salto, ma con un'istruzione diversa (una `select`) eseguita **prima** del salto. Se un guasto fa andare il salto dalla parte opposta alla condizione, la firma "racconta" un percorso diverso da quello effettivo e il controllo all'ingresso se ne accorge. È per questo che RASM rileva i salti invertiti e CFCSS no.

RASM non vede invece un guasto che altera la condizione `c` stessa prima che venga usata: in quel caso select e salto sono d'accordo tra loro. Ma quello è un guasto nei dati, ed è il lavoro di EDDI. Per questo le due famiglie si combinano bene.

In ASPIS ci sono anche due varianti di RASM che non usiamo nella campagna: **inter-RASM**, che estende il controllo attraverso chiamate e ritorni di funzione, e **RACFED** (Vankeirsbilck e altri, 2018), che aggiorna la firma anche dopo ogni istruzione del blocco, per rilevare i salti nel mezzo di un blocco.

4.5 Combinare dati e flusso

ASPIS permette di applicare una tecnica di protezione dei dati e una di controllo del flusso allo stesso programma (`--eddi --rasm`). La protezione dei dati viene applicata per prima, quella del flusso dopo, sul codice già duplicato: quindi anche i blocchi creati da EDDI per i suoi controlli ricevono firme e controlli di RASM. Proprio in questa interazione si nascondeva il terzo bug (capitolo 8).

Parte II. Il lavoro fatto

5. ASPIS in pratica

5.1 Il repository di ASPIS

ASPIS (*Automatic Software-based Protection and Integrity Suite*) è sviluppato dal gruppo HEAPLab del Politecnico (articolo di riferimento: Baroffio, Reghenzani, Fornaciari, ACM TODAES 2024) ed è su GitHub in `HEAPLab/ASPIS`. Abbiamo lavorato sul commit `b66a090`, l'ultimo del branch principale quando è iniziato il progetto. Le parti che ci interessano:

Percorso	Contenuto
`aspis.sh`	lo script che compila un programma applicando le protezioni
`passes/EDDI.cpp`	EDDI e le sue varianti (sEDDI, FDSC, REDDI): circa 2000 righe
`passes/FuncRetToRef.cpp`	trasforma i valori di ritorno in parametri puntatore
`passes/DuplicateGlobals.cpp`	duplica le variabili globali
`passes/CFCSS.cpp`, `RASM.cpp`, `RACFED.cpp`	le tecniche di controllo del flusso (inter-RASM è RASM compilato con un'opzione)
`passes/ASPIS.h`	le dichiarazioni delle classi dei pass
`libs/TypeDeductionAnalysis/`	la libreria che deduce i tipi puntati (un *submodule* git)
`testing/`	il sistema di test: `test.py`, `config/tests.toml`, `tests/` con i programmi

5.2 Compilare ASPIS

Serve LLVM 21.1.8 (scaricato già compilato), CMake e Ninja:

```
git clone --recursive https://github.com/HEAPLab/ASPIS
cd ASPIS && git checkout b66a090
cmake -B build -G Ninja -DLLVM_DIR=<llvm>/lib/cmake/llvm
cmake --build build
```

Il risultato sono le librerie dei pass in `build/passes/`: `libEDDI.so`, `libSEDDI.so`, `libFDSC.so`, `libREDDI.so`, `libCFCSS.so`, `libRASM.so`, `libINTER_RASM.so`, `libRACFED.so`. La compilazione richiede circa un minuto.

5.3 Usare ASPIS

```
./aspis.sh --llvm-bin <llvm>/bin --eddi --rasm programma.c -o programma
```

Le opzioni principali: una tecnica dei dati (`--no-dup`, `--eddi`, `--seddi`, `--fdsc`, `--reddi`, con eventuali `--multiple-errbb` e `--coarse-grained`), una tecnica del flusso (`--no-cfc`, `--cfcss`, `--rasm`, `--inter-rasm`, `--racfed`), `--build-dir` per la cartella dei file intermedi e `--no-cleanup` per non cancellarli alla fine. Con `--no-cleanup` nella cartella resta `out.ll`, l'IR finale: è il file da guardare quando qualcosa non funziona.

Il programma deve definire le due funzioni che ASPIS chiama quando un controllo fallisce:

```c
void DataCorruption_Handler(void) {     /* controlli sui dati (EDDI & co.) */
    fprintf(stderr, "FAULT_DETECTED: DataCorruption\n");
    exit(2);
}
void SigMismatch_Handler(void) {        /* controlli sulle firme (CFC) */
    fprintf(stderr, "FAULT_DETECTED: SigMismatch\n");
    exit(3);
}
```

5.4 La pipeline di `aspis.sh`

Questi sono, nell'ordine, i comandi che `aspis.sh` esegue (semplificati):

```
# 1. ogni sorgente diventa IR, senza ottimizzazioni
clang file.c -S -emit-llvm -O0 -Xclang -disable-O0-optnone -o file.ll
# 2. tutti i file IR vengono uniti in un solo modulo
llvm-link *.ll -o out.ll
# 3. preparazione: via le informazioni di debug, switch trasformati in br
opt --passes="strip" out.ll -o out.ll
opt --passes="lower-switch" out.ll -o out.ll
# 4. i valori di ritorno diventano parametri (vedi 5.5)
opt -load-pass-plugin=libEDDI.so --passes="func-ret-to-ref" out.ll -o out.ll
# 5. protezione dei dati: libEDDI.so, libSEDDI.so, libFDSC.so o libREDDI.so
opt -load-pass-plugin=libEDDI.so --passes="eddi-verify" out.ll -o out.ll
# 6. semplificazione del grafo dei blocchi
opt --passes="simplifycfg" out.ll -o out.ll
# 7. controllo del flusso: cfcss-verify, rasm-verify o racfed-verify
opt -load-pass-plugin=libRASM.so --passes="rasm-verify" out.ll -o out.ll
# 8. coerenza delle variabili globali duplicate
opt -load-pass-plugin=libEDDI.so --passes="duplicate-globals" out.ll -o out.ll
# 9. back-end: IR -> eseguibile
clang out.ll -o programma
```

Due osservazioni:

- l'IR viene prodotto con `-O0` e **nessun passo di ottimizzazione** viene eseguito dopo le protezioni. È voluto: un ottimizzatore riconoscerebbe che le due copie calcolano la stessa cosa e le fonderebbe, cancellando la protezione. Il prezzo è un codice lento, con ogni variabile sullo stack;
- la protezione del flusso viene applicata **dopo** quella dei dati, quindi lavora sul codice già duplicato, compresi i blocchi creati da EDDI.

5.5 FuncRetToRef

Una funzione duplicata deve restituire **due** risultati, uno per copia, ma in C (e in LLVM) una funzione restituisce un solo valore. `FuncRetToRef` risolve il problema prima di EDDI: per ogni funzione `int f(int x)` crea una versione `void f_ret(int x, int *ret)` che scrive il risultato attraverso un puntatore, e sostituisce le chiamate. EDDI poi duplica anche il puntatore di ritorno, così le due copie del risultato finiscono in due posti diversi.

5.6 Come EDDI trasforma il codice

Per ogni funzione `f` del programma, EDDI crea una versione `f_dup` che riceve **ogni parametro due volte** (prima le copie duplicate, poi gli originali) e dentro la quale ogni istruzione è eseguita due volte. Mantiene una mappa

"istruzione originale — istruzione copia". Ogni variabile globale `g` ottiene una copia `g_dup`.

Ai punti di sincronizzazione (store, salti condizionali, chiamate) la funzione `addConsistencyChecks` (in `EDDI.cpp`):

1. spezza il basic block prima dell'istruzione da proteggere;
2. inserisce un nuovo blocco chiamato **`VerificationBB`** con i confronti tra le due copie degli operandi (`icmp eq` per gli interi, `fcmp ueq` per i numeri in virgola mobile, confronto dei valori puntati per i puntatori);
3. termina il blocco con un salto: se tutti i confronti sono veri si prosegue con l'istruzione, altrimenti si va in **`ErrBB`**, un blocco che chiama `DataCorruption_Handler`.

Questo è l'esempio del report: `int scale(int v) { return v * g; }`, con `g` globale, compilato con EDDI (e le nostre correzioni):

```
define void @scale_ret_dup(i32 %0, ptr %1, i32 %2, ptr %3) {
VerificationBB:
  %4 = alloca i32                         ; v
  %5 = alloca i32                         ; la sua copia
  store i32 %2, ptr %4
  store i32 %0, ptr %5
  %6 = load i32, ptr %4
  %7 = load i32, ptr %5
  %8 = load i32, ptr @g
  %9 = load i32, ptr @g_dup               ; la copia della globale
  %10 = mul nsw i32 %6, %8
  %11 = mul nsw i32 %7, %9                ; il calcolo è fatto due volte
  %12 = icmp eq i32 %10, %11              ; controllo di consistenza
  br i1 %12, label %13, label %ErrBB
13:
  store volatile i32 %10, ptr %3          ; "restituisce" le due copie
  store volatile i32 %11, ptr %1
  ret void
ErrBB:
  call void @DataCorruption_Handler()
  unreachable
}
```

Si riconoscono tutti gli ingredienti: il nome `scale_ret_dup` (prima `FuncRetToRef` ha creato `scale_ret`, poi EDDI `scale_ret_dup`), i quattro parametri (`%0` e `%2` sono le due copie di `v`, `%1` e `%3` i due puntatori di ritorno), le operazioni raddoppiate, il confronto prima delle store che fanno uscire il risultato, il blocco `ErrBB`. Il primo blocco si chiama `VerificationBB` perché EDDI lo ha creato spezzando il blocco d'ingresso.

Le chiamate. Se la funzione chiamata ha una versione `_dup` (perché è nel programma ed è stata duplicata), EDDI chiama quella, passando entrambe le copie degli argomenti. Se invece la funzione è esterna (`printf`, `memcpy`, `sin` della libreria C), non può essere duplicata: EDDI la chiama una volta sola con gli argomenti originali. Ma una funzione esterna può **scrivere** attraverso i puntatori che riceve (pensate a `memcpy(dst, src, n)` o `scanf`), aggiornando solo la copia originale dei dati. Per questo, dopo la chiamata, EDDI **risincronizza** la memoria: copia i valori puntati dagli argomenti dalla versione originale a quella duplicata. È la funzione `fixFuncValsPassedByReference`, dove stava il secondo bug.

I puntatori. Le due copie di un puntatore possono legittimamente avere valori diversi: se `p` punta a `x`, la sua copia punta a `x_dup`. Quindi confrontare gli indirizzi non ha senso: EDDI confronta i **valori puntati** (`*p` con `*p_dup`), e se sono a loro volta puntatori scende ancora, fino a un valore non puntatore. Per sapere di che tipo è il valore pun-

tato (con i puntatori opachi, vedi 2.6) usa i tipi dedotti da TypeDeductionAnalysis. È in questo codice (`compareP-trs` e `compareValues`) che stava il primo bug.

5.7 Le altre parti

- **DuplicateGlobals** chiude il lavoro sulle variabili globali: le globali scritte da funzioni non duplicate vengono mantenute coerenti con le loro copie;
- le **annotazioni** permettono di scegliere cosa proteggere: `__attribute__((annotate("exclude")))` lascia una funzione o una globale senza protezione, `to_duplicate` duplica le chiamate a una funzione invece del suo corpo, `to_harden` indica cosa deve proteggere REDDI. Le nostre applicazioni non usano annotazioni: vogliamo proteggere tutto;
- EDDI, a fine compilazione, stampa `Comparison Counter: N`, il numero di controlli inseriti; lo usiamo come misura della quantità di controlli.

5.8 Come si vede RASM nel codice

RASM tiene la firma di esecuzione in una variabile sullo stack (`%2` nell'esempio sotto), letta e scritta con accessi `volatile` perché il compilatore non li elimini. Per ogni blocco `B` crea un blocco `RASM_Verification_BB`, lo mette davanti a `B` e ci fa arrivare tutti i salti che andavano in `B`: lì sottrae il correttivo e controlla la firma. Alla fine di ogni blocco aggiunge l'aggiornamento con la `select` (capitolo 4.4).

Questo è un estratto di una piccola funzione compilata con `--eddi --rasm` (`void clamp(int v) { if (v > 10) g = 10; else g = v; }`):

```
14:
  %15 = select i1 %8, i32 -3, i32 -5          ; correttivo scelto con la condizione
  %16 = load volatile i32, ptr %2             ; firma di esecuzione
  %17 = sub i32 %16, %15
  store volatile i32 %17, ptr %2              ; firma aggiornata
  br i1 %8, label %RASM_Verification_BB2, label %RASM_Verification_BB3

RASM_Verification_BB2:                        ; ingresso del ramo "vero"
  %18 = load volatile i32, ptr %2
  %19 = sub i32 %18, 1                        ; S = S - sub(B)
  store volatile i32 %19, ptr %2
  %20 = icmp eq i32 %19, 24                   ; S == sig(B) ?
  br i1 %20, label %VerificationBB2, label %ErrBB1

VerificationBB2:                              ; blocco creato da EDDI per la store
  store i32 10, ptr @g
  store i32 10, ptr @g_dup
  ...
```

Si vede bene l'intreccio delle due protezioni: il salto condizionale porta prima al controllo di RASM, poi al blocco `VerificationBB2` creato da EDDI. Nel capitolo 8 vedremo cosa succedeva prima della correzione del terzo bug.

6. Le quattro applicazioni

6.1 Perché queste e con quali regole

La proposta lascia libera la scelta del codice da proteggere. Abbiamo scritto quattro programmi piccoli ma realistici, scelti per coprire tipi di calcolo diversi e tipici dei sistemi embedded, dove si usa la SIHFT: crittografia (operazioni su byte e tabelle), elaborazione del segnale (virgola mobile a doppia precisione), compressione (strutture dati e salti che dipendono dai dati), controllo automatico (virgola mobile a singola precisione, stato globale).

Tutti seguono le stesse regole, ciascuna con un motivo preciso:

Regola	Perché
deterministici: nessun input esterno, nessun numero casuale vero	ogni esecuzione senza guasti deve stampare esattamente lo stesso output, altrimenti non si potrebbe confrontare l'output con quello di riferimento
autoverificanti: confrontano i risultati con valori attesi e stampano `SUCCESS` o `FAIL` (codice d'uscita 0 o 1)	un risultato sbagliato si vede sia nell'output sia nel codice d'uscita
definiscono i due **gestori d'errore**, che stampano `FAULT_DETECTED: ...` su stderr ed escono con codice 2 o 3	un rilevamento si riconosce subito, sia nei test sia nella fault injection
sono validi sia in **C sia in C++**	il sistema di test di ASPIS compila il programma di riferimento con `clang++`
sono **brevi**: meno di un milione di istruzioni eseguite	migliaia di esperimenti di fault injection devono durare minuti, non giorni
usano solo la **libreria standard** e sono rilasciati con **licenza MIT**	possono entrare nel repository di ASPIS

Ogni file inizia con un commento che spiega cosa fa il programma e quali caratteristiche del C mette alla prova.

6.2 `aes128.c`: cifratura AES-128

315 righe, 645.680 istruzioni eseguite. Implementa l'algoritmo di cifratura a blocchi AES con chiave di 128 bit (standard FIPS-197):

- calcola all'avvio le tabelle `sbox` e `inv_sbox` (la sostituzione di byte di AES) con l'aritmetica nel campo finito GF(2^8), e le salva in array globali **non costanti**, così ASPIS deve duplicarli;
- esegue il *known-answer test* dell'appendice C.1 dello standard: cifra un blocco noto con una chiave nota e confronta il risultato con quello pubblicato (`69c4e0d86a7b0430d8cdb78070b4c55a`), poi lo decifra;
- cifra e decifra in modalità CBC un messaggio di 64 byte e verifica che il testo decifrato sia uguale all'originale; stampa anche un hash (FNV-1a) del testo cifrato.

Stampa le tabelle e i risultati intermedi e finisce con `SUCCESS` se tutti i confronti tornano. La funzione `print_hex(const char *label, ...)` riceve un'etichetta come parametro e la stampa con `printf("%s", label)`: è proprio il caso che faceva andare in crash EDDI (capitolo 8).

6.3 `fft.c`: trasformata di Fourier

176 righe, 367.763 istruzioni. Calcola la FFT radix-2 (algoritmo di Cooley-Tukey iterativo) di un segnale di 256 campioni complessi in doppia precisione:

- il segnale è la somma di tre sinusoidi (frequenze 5, 12 e 40) più un rumore deterministico;

- trova le tre frequenze dominanti e controlla che siano 5, 12 e 40;
- verifica il **teorema di Parseval** (l'energia del segnale nel tempo è uguale a quella in frequenza, divisa per N);
- calcola la trasformata inversa e controlla che ricostruisca il segnale con un errore inferiore a 10^-9.

Usa la libreria matematica (`sin` , `cos` , `sqrt`), e infatti va compilato con `-lm` .

6.4 `huffman.c` : compressione di Huffman

215 righe, 170.978 istruzioni. Comprime e decomprime un testo fisso con la codifica di Huffman:

- conta le frequenze dei caratteri;
- costruisce l'albero di Huffman con un **min-heap** binario (con un criterio deterministico per i casi di parità);
- assegna i codici con una visita ricorsiva dell'albero;
- scrive il testo compresso in un buffer di bit globale e poi lo decodifica percorrendo l'albero;
- controlla che il testo decodificato sia uguale all'originale e stampa statistiche e un checksum (Adler-32) dei dati compressi.

L'albero è un array di struct con indici interi al posto dei puntatori, e i cicli del heap hanno uscite che dipendono dai dati: tanti salti condizionali "interessanti" per il controllo del flusso.

6.5 `pid_control.c` : controllore PID

152 righe, 477.393 istruzioni. Simula per 2000 passi di 10 ms un motore in corrente continua (modello del primo ordine) controllato da un **PID**:

- il riferimento di velocità cambia a gradini (10, 25, 5, 15 rad/s);
- a metà simulazione si applica un disturbo di coppia;
- il sensore di velocità è quantizzato come un encoder;
- il PID ha la derivata filtrata, la saturazione dell'uscita a ±24 V e un meccanismo *anti-windup* (l'integrale viene aggiornato solo se aiuta a uscire dalla saturazione).

Stampa ogni 250 passi lo stato del sistema e alla fine l'errore integrale assoluto (IAE), la sovraelongazione e i passi in saturazione; il controllo finale verifica che la velocità abbia raggiunto il riferimento. Usa `float` (singola precisione), tipico dei microcontrollori, e tiene lo stato del controllore in una struct globale aggiornata tramite puntatore.

7. I test di compatibilità

7.1 Il sistema di test di ASPIS

ASPIS ha un sistema di test basato su **pytest** (cartella `testing/`). I programmi di test sono elencati in `testing/config/tests.toml`, uno per voce:

```
[[tests]]
test_name = "c_fft"
source_file = "c/applications/fft.c"     # relativo a testing/tests/
add_compiler_flags = "-lm"               # opzioni in più per aspis.sh
# black_list = ["--inter-rasm"]          # tecniche da non provare
```

Il file `testing/test.py` prende ogni test e lo compila con **55 combinazioni**: le 11 opzioni dei dati (`--no-dup`, `--eddi`, `--eddi --multiple-errbb`, `--eddi --coarse-grained`, `--eddi --multiple-errbb --coarse-grained`, le stesse quattro per `--reddi`, `--seddi`, `--fdsc`) per le 5 del flusso (`--no-cfc`, `--cfcss`, `--rasm`, `--racfed`, `--inter-rasm`). Per ogni combinazione:

1. compila il programma con `clang++` senza ASPIS (una volta sola) e lo esegue: è l'output di riferimento;
2. compila il programma con `aspis.sh` e le opzioni della combinazione;
3. lo esegue; il test **passa** se l'esecuzione termina con codice 0 e l'output è uguale a quello di riferimento.

Ogni combinazione ha un limite di 60 secondi. Le combinazioni nella `black_list` di un test vengono saltate (*skipped*).

Si lancia così, dalla cartella `testing/` di ASPIS (dopo aver scritto il percorso di LLVM in `testing/config/llvm.toml`):

```
pytest test.py -v > harness.log
```

Ogni riga del log è una combinazione, per esempio `test.py::test_aspis[c_fft---rasm---eddi] PASSED`: nome del test, tecnica del flusso, tecnica dei dati, esito.

7.2 Cosa abbiamo fatto

Abbiamo aggiunto le quattro applicazioni in `testing/tests/c/applications/` e le loro voci in fondo a `tests.toml` (sono nel file `aspis-integration/flc_tests.toml`), poi abbiamo eseguito l'intera suite due volte:

- sul branch principale di ASPIS (`b66a090`): log in `results/compatibility/harness_master.log.gz`;
- con le nostre tre correzioni: log in `results/compatibility/harness_fixed.log.gz`.

Ognuna delle due esecuzioni compila e prova 52 test × 55 combinazioni = 2860 programmi e dura circa 9 minuti. Lo script `fault_injection/analyze.py` legge i log e produce le tabelle del report.

7.3 I risultati

Le nostre applicazioni (4 applicazioni × 5 tecniche del flusso per ogni riga):

Protezione dei dati	`master`	con le correzioni
nessuna (`--no-dup`)	20 / 20	20 / 20
EDDI, 4 varianti	0 / 80	80 / 80
sEDDI	0 / 20	20 / 20
FDSC	0 / 20	20 / 20
REDDI, 4 varianti	80 / 80	80 / 80
totale	**100 / 220**	**220 / 220**

Come si legge:

- **le tecniche del flusso da sole funzionano** su tutte le applicazioni, anche su `master` (riga `--no-dup`);
- **REDDI passa**, ma per un motivo banale: REDDI protegge solo ciò che è annotato con `to_harden`, e le nostre applicazioni non hanno annotazioni, quindi REDDI non protegge nulla;
- **EDDI, sEDDI e FDSC vanno in crash su tutte e quattro le applicazioni** (*segmentation fault*), con qualunque tecnica del flusso. Il capitolo 8 spiega perché;
- con le correzioni passano tutte le 220 combinazioni.

I 48 test che ASPIS aveva già:

Esito	`master`	con le correzioni
passati	2453	2453
falliti	33	33
saltati	154	154

I risultati sono **identici**: le correzioni non rompono nulla (nessuna *regressione*). I 154 saltati sono le combinazioni nelle `black_list`. I 33 falliti sono tutte le combinazioni non escluse di un solo test, `cpp_complex`, e falliscono anche su `master` già nel primo passo: il programma di riferimento non compila, perché il sorgente usa `std::powf`, che la libreria C++ della nostra macchina (libstdc++ 13) non dichiara. È un problema del test e dell'ambiente, non di ASPIS né delle nostre modifiche.

8. I tre bug e le correzioni

8.1 Come si cerca un bug in un pass

Quando un programma protetto da ASPIS non funziona, il metodo è sempre lo stesso:

1. **riprodurre**: trovare la combinazione di opzioni che fallisce (il log del sistema di test la dice) e ripetere la compilazione a mano con `aspis.sh ... --no-cleanup`, così restano i file intermedi;
2. **capire dove**: se il programma va in crash, eseguirlo con `gdb` (`run`, poi `bt` per lo stack delle chiamate e `x/10i $pc` per le istruzioni vicine); se il compilatore si ferma con un errore, leggere il messaggio di `opt`; poi guardare l'IR in `out.ll` intorno al punto incriminato;
3. **ridurre**: scrivere un programma minimo, di poche righe, che mostra lo stesso problema (un *riproduttore*). Serve a capire la causa e a verificare la correzione;
4. **leggere il codice del pass** che produce l'IR sbagliato e capire perché;
5. **correggere**, ricompilare ASPIS (`cmake --build build`) e verificare: il riproduttore deve funzionare, e **tutti** i test di ASPIS devono dare gli stessi risultati di prima.

8.2 Bug 1: i controlli che leggono da un puntatore nullo

Sintomo. Il riproduttore `aspis-integration/reproducers/null_pointer.c`:

```
int main(void) {
    int *v = NULL;
    v = realloc(v, 4 * sizeof(int));   /* realloc(NULL, n) equivale a malloc(n) */
    v[0] = 42;
    printf("%d\n", v[0]);
    free(v);
    return 0;
}
```

compilato con `--eddi` va in *segmentation fault* sul branch principale. Questo bug non colpisce direttamente le nostre quattro applicazioni, ma la correzione del bug 2 (sezione 8.3) deve leggere memoria attraverso puntatori che possono essere nulli, e quindi ha bisogno di questa: le due patch vanno insieme.

Causa. Prima di una chiamata EDDI controlla gli argomenti, e per un puntatore confronta i **valori puntati** (vedi 5.6). Il codice (`comparePtrs` in `EDDI.cpp`) caricava `*v` e `*v_dup` senza chiedersi se `v` fosse nullo: con `realloc(NULL, ...)` il controllo stesso dereferenzia il puntatore nullo e il programma va in crash, anche senza nessun guasto. Lo stesso succede alla fine di una lista concatenata, o con `free(NULL)`.

Correzione (patch 0001). Una nuova funzione, `getNullSafePointer`, restituisce un puntatore da cui si può sempre leggere:

```
// se Ptr è nullo, legge da una costante globale piena di zeri
Value *EDDI::getNullSafePointer(Value *Ptr, Type *Ty, IRBuilder<> &B) {
  GlobalVariable *&Guard = NullGuards[Ty];          // una costante per tipo
  if (Guard == nullptr)
    Guard = new GlobalVariable(..., Constant::getNullValue(Ty), "aspis_null_guard");
  return B.CreateSelect(B.CreateIsNull(Ptr), Guard, Ptr);   // p == NULL ? guard : p
}
```

Nell'IR prodotto ogni lettura di un controllo diventa:

```
%isnull = icmp eq ptr %p, null
%safe   = select i1 %isnull, ptr @aspis_null_guard, ptr %p
%val    = load i32, ptr %safe
```

Se entrambe le copie del puntatore sono nulle, si leggono due zeri e il controllo passa (giusto: le due copie sono coerenti). Se solo una è nulla (per esempio per un guasto), si legge zero da una parte e il valore vero dall'altra: i valori differiscono e il guasto viene **rilevato**.

Nella stessa patch abbiamo sistemato anche `compareValues`, la funzione che sceglie come confrontare due valori. Prima la scelta dipendeva dal tipo *dedotto*, cioè quello del sorgente C; ma il compilatore a volte rappresenta un valore con un tipo diverso: una piccola struct passata per valore diventa un `i64`. Il vecchio codice la trattava da struct ed estraeva i campi da un intero, facendo andare in crash il compilatore. Ora il confronto dipende dal **tipo LLVM** del valore: `icmp eq` per gli interi, `fcmp ueq` per la virgola mobile, confronto dei valori puntati per i puntatori. I valori aggregati (struct e array interi) **non** vengono confrontati: il vecchio codice generava confronti senza effetto, e comunque il loro *padding* (i byte di riempimento tra i campi) può legittimamente differire tra le due copie.

8.3 Bug 2: la copia dopo le chiamate di libreria

Sintomo. È il bug che faceva andare in crash **tutte e quattro** le applicazioni con EDDI, sEDDI e FDSC. Con `gdb` si vede che il crash è sempre una scrittura di 8 byte (`mov %rcx,(%rax)`) subito dopo una chiamata a `printf`, e che l'indirizzo scritto è una **stringa letterale**, che sta in memoria di sola lettura (la sezione `.rodata`):

Applicazione	Stringa sovrascritta	Codice C
`aes128`	`"kat ct: "`	`print_hex("kat ct: ", ...)`, che fa `printf("%s", label)`
`fft`	`"OK"`	`printf("ifft reconstruction: %s\n", max_err < 1e-9 ? "OK" : "MISMATCH")`
`huffman`	`"OK"`	`printf("roundtrip: %s\n", ok ? "OK" : "MISMATCH")`
`pid_control`	`"SUCCESS"`	`printf("%s\n", ok ? "SUCCESS" : "FAIL")`

Il riproduttore `reproducers/string_literal.c` è di cinque righe:

```
static void show(const char *label) { printf("%s\n", label); }
int main(void) { show("hello"); return 0; }
```

Causa. Ricordate la risincronizzazione dopo le chiamate esterne (5.6): dopo `printf`, EDDI copia i valori puntati dagli argomenti dalla versione originale a quella duplicata. Il vecchio codice di `fixFuncValsPassedByReference` faceva, per ogni argomento che fosse il risultato di un'istruzione e avesse una copia:

```
// vecchio codice (semplificato)
Value *Original = argomento, *Copy = la sua copia;
Instruction *TmpLoad  = B.CreateLoad(Original->getType(), Original);  // legge un ptr: 8 byte
Instruction *TmpStore = B.CreateStore(TmpLoad, Copy);                 // li scrive nella copia
```

I problemi sono tre:

1. **la dimensione è sbagliata**: legge e scrive sempre un valore di tipo puntatore, cioè 8 byte, qualunque sia il tipo puntato. Per un `int` (4 byte) scrive oltre la fine della copia; per un buffer di 100 byte ne copia solo 8;
2. **non distingue la memoria condivisa**: quando l'argomento è una stringa letterale, la "copia" del puntatore punta **alla stessa stringa**: nel caso di `show("hello")` il parametro `label` e la sua copia valgono entrambi

l'indirizzo di `"hello"`; nel caso di `ok ? "SUCCESS" : "FAIL"` l'istruzione `select` e la sua copia scelgono la stessa costante. Copiare da una a sull'altra non servirebbe a nulla, ma soprattutto **scrive in memoria di sola lettura**, e il programma va in crash;

3. **copia anche quando non serve**: `printf` non scrive mai attraverso i suoi argomenti.

Correzione (patch 0002). La funzione è stata riscritta. Per ogni argomento puntatore di una chiamata a una funzione non duplicata:

1. se la funzione (o quell'argomento) **non può scrivere** in memoria (`onlyReadsMemory`, un'informazione che LLVM conosce per molte funzioni di libreria e per i parametri `readonly`), non si fa nulla;
2. si risale all'**oggetto** a cui punta l'argomento (`getUnderlyingObject`: una variabile locale, una globale, o "sconosciuto"); se l'oggetto è una **globale costante** (come una stringa letterale) si salta;
3. se l'originale e la copia sono due **variabili distinte di dimensione nota**, si copia l'intero oggetto con un `memcpy` della dimensione giusta;
4. altrimenti (memoria dinamica, puntatori ricevuti come parametro) si usa il tipo dedotto da TDA per sapere **quanti byte** copiare, e si genera codice che controlla **a tempo di esecuzione** se i due puntatori sono uguali (o se la copia è nulla): in quel caso la memoria è condivisa e la copia viene dirottata su un buffer di appoggio (*scratch*) sullo stack, dove non fa danni:

```
Value *Shared = B.CreateOr(B.CreateICmpEQ(Original, Duplicate),   // stessi indirizzi?
                           B.CreateIsNull(Duplicate));            // copia nulla?
Value *Dest   = B.CreateSelect(Shared, Scratch, Duplicate);       // se sì, nel buffer
Value *Source = getNullSafePointer(Original, BytesTy, B);         // lettura sicura (bug 1)
B.CreateMemCpy(Dest, ..., Source, ..., Size);                     // la dimensione giusta
```

Notate l'uso di `getNullSafePointer`: la copia legge attraverso il puntatore originale, che potrebbe essere nullo. È per questo che la patch 0002 dipende dalla 0001 e le due vanno applicate insieme.

8.4 Bug 3: RASM non controllava i blocchi creati da EDDI

Questo bug **non fa fallire nessun programma**: i test di compatibilità non potevano trovarlo. L'ha trovato la fault injection.

Sintomo. Ci si aspetta che EDDI+RASM protegga almeno quanto RASM da solo, visto che aggiunge altri controlli. Invece, contro i salti invertiti, EDDI+RASM lasciava il **10,5%** di SDC, contro lo **0,73%** di RASM da solo (capitolo 11).

Come l'abbiamo trovato. Il file dei risultati grezzi (`results/fault_injection/before_fix/raw.csv.gz`) ha una riga per esperimento, con l'indirizzo dell'istruzione colpita (`pc_off`, la distanza dall'inizio del codice). Prendendo gli esperimenti finiti in SDC e disassemblando il programma a quegli indirizzi (con `objdump -d`), si vede che **tutte** le 421 SDC sono avvenute su salti condizionali i cui due successori cominciano con la stessa istruzione: `mov DWORD PTR [rbp-0x4], <costante>`, cioè la scrittura di una costante nella variabile della firma di RASM. In altre parole, in quei blocchi RASM non controllava la firma: la reimpostava. Lo script `fault_injection/sdc_sites.py` fa esattamente questa analisi:

```
$ python3 fault_injection/sdc_sites.py results/fault_injection/before_fix/raw.csv.gz \
      <campagna> huffman EDDI+RASM
huffman EDDI+RASM branch: 139 SDC at 17 instructions
    33 x  b362  fn_2_ret_dup         jne    b369 <fn_2_ret_dup+0x279>
            not taken -> jmp    b440 <fn_2_ret_dup+0x350>
            taken     -> mov    DWORD PTR [rbp-0x4],0x46c
    ...
```

Causa. In `RASM.cpp`, all'ingresso di ogni blocco, c'è questa scelta:

```
// codice originale
if (isa<LandingPadInst>(BB.getFirstNonPHI()) ||
    BB.getName().contains_insensitive("verification")) {
  // NON controlla la firma: la reimposta al valore atteso del blocco
  BChecker.CreateStore(ConstantInt::get(IntType, randomNumberBB), &RuntimeSig, true);
} else {
  // crea il blocco RASM_Verification_BB che controlla la firma
}
```

L'eccezione serviva per i blocchi creati da RASM stesso (che si chiamano `RASM_Verification_BB...`), dove la firma non va controllata due volte. Ma il test sul nome (`contains_insensitive("verification")`, cioè "il nome contiene *verification*, maiuscole o minuscole") è troppo largo: vale anche per i blocchi `VerificationBB` creati da EDDI. E con EDDI questi blocchi sono dappertutto, perché EDDI ne crea uno prima di ogni store, salto e chiamata: sono le destinazioni di quasi tutti i salti.

Ecco la stessa funzione del capitolo 5.8 (`clamp`) compilata **prima** della correzione:

```
  br i1 %8, label %VerificationBB2, label %VerificationBB3   ; salto condizionale

VerificationBB2:                    ; nessun controllo di RASM davanti
  store volatile i32 24, ptr %2     ; firma REIMPOSTATA al valore atteso, senza controllo
  store i32 10, ptr @g
  ...
VerificationBB3:
  store volatile i32 26, ptr %2     ; idem
  ...
```

Se il salto viene invertito, il programma entra nel blocco sbagliato, ma la prima cosa che fa è sovrascrivere la firma con il valore "giusto" per quel blocco: l'errore viene cancellato prima di poter essere controllato. Con la correzione (capitolo 5.8) il salto va prima in `RASM_Verification_BB2` o `RASM_Verification_BB3`, che controllano la firma.

Correzione (patch 0003). La condizione diventa `BB.getName().starts_with("RASM_")`: l'eccezione vale solo per i blocchi creati da RASM stesso (e per i *landing pad*, i blocchi di gestione delle eccezioni del C++). Lo stesso errore c'era, identico, in `RACFED.cpp` (con `starts_with("RACFED_")`), e la patch corregge anche quello.

Verifica. Abbiamo rifatto gli esperimenti di EDDI+RASM con la correzione: nessuna SDC sui 4000 salti invertiti (prima erano 421), e i controlli di RASM ne rilevano il 66,0% invece del 34,6% (il resto lo rilevano i controlli di EDDI). La tabella completa è nel capitolo 11.

9. L'iniettore di guasti (`fi.c`)

`fault_injection/injector/fi.c` è un programma C di circa 550 righe che si compila con `gcc -O2 -o fi fi.c` e ha due comandi: `fi profile` (esecuzione di riferimento) e `fi inject` (gli esperimenti).

9.1 `ptrace` in breve

`ptrace` è la chiamata di sistema di Linux con cui un processo (il *tracer*, qui `fi`) controlla un altro processo (il *tracee*, il programma sotto test): è quella che usa `gdb`. Le operazioni che usiamo:

Operazione	Cosa fa
`PTRACE_TRACEME`	chiamata dal figlio prima di `execve`: "fatti controllare da mio padre". Il figlio si ferma subito dopo l'`exec`
`waitpid`	il padre aspetta che il figlio si fermi (per un segnale) o termini
`PTRACE_GETREGS` / `PTRACE_SETREGS`	legge / scrive tutti i registri del figlio (`struct user_regs_struct`: `rax`, …, `rip`, `eflags`)
`PTRACE_PEEKTEXT` / `PTRACE_POKETEXT`	legge / scrive 8 byte della memoria del figlio (anche il codice)
`PTRACE_SINGLESTEP`	esegue una sola istruzione e si ferma di nuovo
`PTRACE_CONT`	fa ripartire il figlio fino al prossimo segnale
`PTRACE_DETACH`	smette di controllare il figlio, che prosegue da solo
`PTRACE_O_EXITKILL`	se `fi` muore, il figlio viene ucciso (niente processi orfani)

Quando il figlio esegue un'istruzione `int3` (il byte `0xcc`, il *breakpoint* dei debugger) o termina un passo singolo, riceve un `SIGTRAP` e si ferma: il padre lo vede con `waitpid`.

Il figlio viene avviato (funzione `spawn`) con `fork` ed `execve`, con l'output rediretto su file, e con `personality(ADDR_NO_RANDOMIZE)`, che **disattiva la randomizzazione degli indirizzi** (ASLR): così il codice del programma sta allo stesso indirizzo in ogni esecuzione, e l'indirizzo di un'istruzione trovato nel profilo vale anche negli esperimenti.

9.2 `fi profile`: l'esecuzione di riferimento

```
fi profile golden.prof golden.stdout -- ./prog
```

La funzione `cmd_profile`:

1. avvia il programma sotto `ptrace`;
2. legge `/proc/<pid>/maps` per trovare l'intervallo di indirizzi del **codice del programma** (la mappatura eseguibile del file del programma), così da distinguere le istruzioni del programma da quelle della libreria C e del caricatore dinamico (funzione `text_range`);
3. esegue il programma **un'istruzione alla volta** (`PTRACE_SINGLESTEP`). A ogni passo legge `rip`: se è nel codice del programma, incrementa il contatore di quell'indirizzo. La prima volta che incontra un indirizzo, legge i byte dell'istruzione e controlla se è un salto condizionale (`decode_jcc`);
4. alla fine scrive il profilo e salva l'output (il *golden output*).

Il profilo è un file di testo:

```
fi-profile 1
text 555555555000 555555557000     # intervallo del codice del programma
dynamic 645680 774766              # istruzioni eseguite: nel programma, in totale
exit 0                             # codice d'uscita
pcs 1316                           # numero di indirizzi distinti eseguiti
0 1 0 0 0 0                        # offset, esecuzioni, jcc?, cc, lunghezza, spostamento
4 1 0 0 0 0
...
12 1 1 4 2 2                       # un salto condizionale (cc 4 = je), lungo 2 byte
...
```

(sono le prime righe del profilo di `aes128` senza protezione: 645.680 istruzioni nel programma, 774.766 contando anche la libreria C).

Ogni riga dice quante volte è stata eseguita l'istruzione a quell'offset e, se è un salto condizionale, il suo *condition code*, la sua lunghezza in byte e lo spostamento del salto.

Eseguire milioni di istruzioni una per una è lento (ogni passo sono due cambi di contesto tra processi): il profilo dei programmi più grandi richiede decine di secondi. Ma si fa una volta sola per ogni programma.

9.3 Riconoscere un salto condizionale

`decode_jcc` riconosce le due codifiche dei salti condizionali x86:

- `0x70`-`0x7f` + 1 byte di spostamento (salto corto, lunghezza 2);
- `0x0f` `0x80`-`0x8f` + 4 byte di spostamento (salto lungo, lunghezza 6);

eventualmente precedute da un prefisso di predizione (`0x2e` o `0x3e`). Il *condition code* sono i 4 bit bassi dell'opcode. `jcc_taken` calcola, dai flag, se il salto verrebbe preso:

`cc >> 1`	condizione	salti
0	`OF`	`jo` / `jno`
1	`CF`	`jb` / `jae`
2	`ZF`	`je` / `jne`
3	`CF or ZF`	`jbe` / `ja`
4	`SF`	`js` / `jns`
5	`PF`	`jp` / `jnp`
6	`SF != OF`	`jl` / `jge`
7	`ZF or SF != OF`	`jle` / `jg`

Se il bit basso di `cc` è 1 la condizione è negata (`jne` è "salta se **non** ZF").

9.4 `fi inject`: gli esperimenti

```
fi inject golden.prof golden.stdout reg 12345 100 300 /dev/shm/tmp -- ./prog
#                                  modello seme  N  timeout(ms)
```

Per ognuno degli N esperimenti, la funzione `cmd_inject`:

1. **sceglie l'istante** (`pick_target`): estrae un numero `k` tra 0 e il totale delle istruzioni eseguite nel profilo, e scorre la lista degli indirizzi sottraendo i contatori finché `k` cade dentro uno di essi. Così ogni *esecuzione* di ogni

istruzione ha la stessa probabilità: è come scegliere un istante a caso dell'esecuzione. Il risultato è una coppia (indirizzo, occorrenza): "la 37ª esecuzione dell'istruzione all'offset `0x1a2b`". Per il modello `branch` si considerano solo le esecuzioni dei salti condizionali;

2. sceglie a caso anche il registro (uno dei 16) e il bit (0-63) da invertire. I numeri casuali vengono da un generatore *splitmix64* inizializzato con il seme: stesso seme, stessi esperimenti;
3. **avvia** il programma sotto `ptrace`;
4. **lo ferma** subito prima dell'occorrenza scelta (`run_to_target`): scrive `0xcc` (`int3`) all'indirizzo dell'istruzione e lo lascia correre. A ogni `SIGTRAP` controlla che si sia fermato lì, rimette il byte originale, riporta `rip` indietro di uno e conta. Se non è ancora l'occorrenza giusta, esegue l'istruzione originale con un passo singolo, rimette l'`int3` e continua;
5. **inietta il guasto**: legge i registri e
   - modello `reg`: inverte il bit scelto del registro scelto (`*gpr(&regs, reg) ^= 1ull << bit`);
   - modello `branch`: guarda i flag con `jcc_taken`; se il salto verrebbe preso, mette in `rip` l'indirizzo dell'istruzione successiva (il salto "non viene preso"), altrimenti mette la destinazione del salto. Il salto non viene eseguito: il programma riparte direttamente dal successore sbagliato;

   e scrive i registri modificati;
6. **lo lascia andare** (`PTRACE_DETACH`) e aspetta che termini, al massimo per il tempo limite (`wait_with_timeout`, che controlla ogni 100 µs e uccide il programma allo scadere);
7. **classifica** l'esito, in quest'ordine:
   - `hang` se è scaduto il tempo limite;
   - `crash` se il programma è stato ucciso da un segnale;
   - `detected_data` se su stderr c'è `FAULT_DETECTED: DataCorruption`;
   - `detected_cfc` se su stderr c'è `FAULT_DETECTED: SigMismatch`;
   - `masked` se l'output è identico a quello di riferimento e il codice d'uscita è lo stesso;
   - `sdc` altrimenti (output diverso, oppure il programma stesso si è accorto dell'errore e ha stampato `FAIL`);
8. stampa una riga CSV: numero dell'esperimento, modello, offset e occorrenza dell'istruzione colpita, registro (o *condition code*), bit, esito, codice d'uscita, segnale, durata in microsecondi. Per esempio, dai risultati di `aes128` con EDDI:

```
id,model,pc_off,occurrence,target,bit,injected,outcome,exit,signal,time_us
0,reg,5745,384,rsp,18,1,crash,-1,11,17309
1,reg,77bb,1031,rbx,51,1,masked,0,0,35805
```

Il primo esperimento ha invertito il bit 18 di `rsp` (il puntatore allo stack) alla 385ª esecuzione dell'istruzione all'offset `0x5745`: il programma è stato ucciso dal segnale 11 (`SIGSEGV`). Il secondo ha colpito `rbx` senza effetti visibili.

Un esperimento dura in media circa 60 millisecondi: avvio del programma, corsa fino al punto scelto (con due fermate per ogni passaggio sull'istruzione bersaglio, che nei cicli possono essere migliaia), resto dell'esecuzione.

9.5 Cosa l'iniettore non fa

- inietta solo mentre il programma esegue **il proprio codice**, mai nella libreria C (che ASPIS non protegge);
- non tocca i registri **SSE** (`xmm0` …), dove stanno i valori in virgola mobile: per `fft` e `pid_control` i guasti nei calcoli in virgola mobile sono rappresentati solo quando colpiscono indirizzi, indici o interi;
- un solo guasto per esecuzione, niente guasti in memoria.

10. La campagna e la statistica

10.1 Il file di configurazione

`fault_injection/campaign.toml` descrive la campagna:

```toml
[campaign]
root = ".."                                # i sorgenti sono relativi a ASPIS-Project/
output = "~/aspis-project/campaign"        # dove scrivere eseguibili, profili, risultati
aspis = "~/aspis-project/ASPIS"            # ASPIS con le nostre patch
llvm_bin = "~/aspis-project/toolchain/LLVM-21.1.8-Linux-X64/bin"
fi = "~/aspis-project/fi"                  # l'iniettore compilato
models = ["reg", "branch"]
injections = 1000                          # per applicazione x configurazione x modello
chunk = 100                                # esperimenti per ogni processo fi
seed = 2026
timeout_factor = 20                        # hang = 20 volte il tempo normale...
timeout_min_ms = 300                       # ...ma almeno 300 ms

[[config]]                                 # la prima è il riferimento
name = "baseline"
flags = "--no-dup --no-cfc"
...                                        # EDDI, CFCSS, RASM, EDDI+CFCSS, EDDI+RASM

[[application]]
name = "fft"
source = "benchmarks/fft.c"
flags = "-lm"
...
```

I percorsi in `[campaign]` vanno adattati alla propria macchina.

10.2 Cosa fa `campaign.py`

```
python3 fault_injection/campaign.py fault_injection/campaign.toml --jobs 4
```

1. **Compila** ogni applicazione con ogni configurazione (`aspis.sh` con le opzioni della configurazione), ognuna nella sua cartella `build/<applicazione>/<configurazione>/`. `aspis.sh` viene eseguito *dentro* quella cartella perché EDDI scrive un file di lavoro (`compiled_eddi_functions.csv`) nella cartella corrente: due compilazioni in parallelo nella stessa cartella si sovrascriverebbero il file a vicenda.
2. **Misura** la dimensione del codice (sezione `.text`, con `llvm-size`) e il numero di controlli inseriti da EDDI (la riga `Comparison Counter` stampata da `aspis.sh`).
3. **Esegue le golden run** con `fi profile` e controlla che tutte le configurazioni stampino esattamente lo stesso output della `baseline`: una protezione non deve cambiare il risultato.
4. Scrive `metrics.csv`: per ogni applicazione e configurazione, dimensione del codice, controlli, istruzioni eseguite.
5. **Calcola il tempo limite** di ogni programma: esegue il programma 5 volte senza `ptrace`, prende il tempo più breve e lo moltiplica per 20 (minimo 300 ms). Un esperimento che dura di più è un `hang`.
6. **Lancia gli esperimenti** in blocchi (*chunk*) di 100, ognuno con un proprio seme, in parallelo su tutti i core. Il seme di ogni blocco è calcolato con SHA-256 dai nomi (`"2026/aes128/EDDI/reg/300"` → i primi 15 caratteri

esadecimali dell'hash): è sempre lo stesso, quindi la campagna è **riproducibile**, e i blocchi sono indipendenti tra loro.

7. **Aggrega**: conta gli esiti per applicazione, configurazione e modello, calcola la percentuale di SDC con il suo intervallo di confidenza e scrive `outcomes.csv`.

Ogni passo salva il suo risultato su file (scrivendo prima un file `.part` e rinominandolo alla fine): se la campagna si interrompe, rilanciandola rifà solo quello che manca.

I numeri della nostra campagna: 4 applicazioni × 6 configurazioni × 2 modelli × 1000 = **48.000 esperimenti**, in circa 18 minuti su una macchina virtuale con 4 core (6 minuti di golden run e 12 di esperimenti). Più gli **8000** di EDDI+RASM con ASPIS senza la correzione del bug 3, in circa 4 minuti e mezzo.

`export.py` copia `metrics.csv` e `outcomes.csv` nel repository (`results/fault_injection/`) e unisce tutti i CSV dei blocchi in un unico `raw.csv.gz` (una riga per esperimento).

10.3 Percentuali e intervalli di confidenza

Ogni percentuale della campagna è una **stima**: abbiamo scelto 4000 guasti a caso tra tutti quelli possibili, e la percentuale osservata `p = k / n` (`k` SDC su `n` esperimenti) è vicina, ma non uguale, alla vera probabilità. L'**intervallo di confidenza al 95%** è un intervallo costruito in modo che, se si ripetesse la campagna molte volte, conterrebbe la vera probabilità nel 95% dei casi.

La formula più nota (intervallo "di Wald") è `p ± 1,96 · sqrt(p(1-p)/n)`, ma funziona male quando `p` è vicina a 0, proprio il nostro caso: con `k = 0` dà l'intervallo `[0, 0]`, come se fossimo sicuri che la probabilità sia esattamente zero. Usiamo quindi l'**intervallo di Wilson**, che si comporta bene anche agli estremi:

$$\text{intervallo} = \frac{p + z^2/(2n) \;\pm\; z \cdot \sqrt{\,p(1-p)/n + z^2/(4n^2)\,}}{1 + z^2/n} \qquad \text{con } z = 1{,}96$$

Esempi dalla nostra campagna:

	k / n	p	intervallo di Wilson al 95%
EDDI, bit-flip nei registri	6 / 4000	0,15%	0,07% – 0,33%
EDDI+RASM, salti invertiti	0 / 4000	0%	0,00% – 0,10%
baseline, bit-flip nei registri	68 / 4000	1,70%	1,34% – 2,15%

Il secondo esempio si legge: "non abbiamo visto nessuna SDC su 4000 salti invertiti; la vera probabilità è, con il 95% di confidenza, sotto lo 0,10%". Quando due intervalli non si sovrappongono (1,34-2,15% contro 0,07-0,33%), la differenza non è dovuta al caso.

10.4 La media geometrica

Per riassumere il costo su quattro applicazioni usiamo la **media geometrica** dei rapporti, `(r1 · r2 · r3 · r4)^(1/4)`, non quella aritmetica. Il motivo: i rapporti si moltiplicano, non si sommano. Se un programma raddoppia (2x) e un altro dimezza (0,5x), la media aritmetica dice 1,25x, quella geometrica 1x, che è la risposta giusta ("in media non cambia nulla"). Per EDDI i rapporti delle istruzioni eseguite sono 3,82 (`aes128`), 3,63 (`fft`), 3,56 (`huffman`) e 4,15 (`pid_control`): la media geometrica è 3,78.

10.5 Gli script di analisi

- `analyze.py` legge i risultati della campagna e i log del sistema di test e scrive le tabelle (`report/tables/*.md`, in formato Markdown) e le figure (`report/figures/*.svg` e `.png`, con matplotlib) che `report/build.py` inserisce nel report;
- `sdc_sites.py` (capitolo 8.4) mostra dove sono avvenute le SDC di una configurazione: le istruzioni colpite, quante volte, e i successori dei salti.

11. I risultati, spiegati

11.1 Il costo delle protezioni

Configurazione	Dimensione del codice	Istruzioni eseguite	Controlli (EDDI)
baseline	1,00x	1,00x	0
EDDI	9,18x	3,78x	1259
CFCSS	1,54x	1,65x	0
RASM	1,93x	2,33x	0
EDDI+CFCSS	13,56x	5,51x	1259
EDDI+RASM	14,40x	5,99x	1259

(medie geometriche sulle quattro applicazioni, rispetto alla `baseline`; 1259 è il totale dei controlli inseriti nelle quattro applicazioni.)

Cosa dire:

- **EDDI costa circa 3,8 volte** le istruzioni: ogni istruzione è eseguita due volte, e ogni controllo aggiunge confronti e un salto;
- **il codice cresce molto più dell'esecuzione** (9,2x): ogni funzione esiste fino in quattro versioni (`f`, `f_ret`, `f_dup`, `f_ret_dup`), anche quelle che alla fine non vengono più chiamate restano nell'eseguibile, ogni controllo spezza un blocco in più blocchi, e senza ottimizzazioni ogni valore passa dallo stack;
- **le tecniche del flusso costano meno** (1,7x CFCSS, 2,3x RASM): aggiungono poche istruzioni per blocco. RASM costa più di CFCSS perché aggiorna la firma due volte per blocco (all'ingresso e prima dell'uscita, con la `select`);
- **combinate, i costi si sommano**: EDDI+RASM esegue 6 volte le istruzioni dell'originale.

Il costo è misurato in **istruzioni eseguite** (contate dal profilo), non in tempo: i programmi durano pochi millisecondi, troppo poco per misure di tempo affidabili, mentre il numero di istruzioni è esatto e ripetibile.

11.2 Bit-flip nei registri

Configurazione	masked	rilevati (dati)	rilevati (CFC)	crash	hang	**SDC**	intervallo 95%
baseline	86,2	0,0	0,0	12,0	0,1	**1,70**	1,34–2,15
EDDI	84,0	2,9	0,0	13,0	0,0	**0,15**	0,07–0,33
CFCSS	85,5	0,0	2,1	11,5	0,0	**0,88**	0,63–1,21
RASM	85,8	0,0	2,2	11,3	0,0	**0,65**	0,44–0,95
EDDI+CFCSS	83,4	1,3	2,5	12,8	0,0	**0,03**	0,00–0,14
EDDI+RASM	83,3	2,0	1,6	13,1	0,0	**0,03**	0,00–0,14

(percentuali su 4000 guasti per riga.)

Cosa dire:

- **la maggior parte dei bit-flip non fa nulla** (86% *masked*): in un dato istante molti registri non contengono valori che verranno usati, o vengono sovrascritti prima di essere letti;
- **il 12% fa andare in crash il programma**: un bit cambiato in un indirizzo porta a leggere o scrivere fuori dalla memoria valida. Nella `baseline` l'87% dei crash viene da un bit-flip in `rsp` (52%) o in `rbp` (35%), i due pun-

tatori allo stack: con `-O0` ogni variabile si legge e si scrive attraverso di loro;

- **l'1,70% produce un risultato sbagliato in silenzio**: sono questi i guasti che contano;
- **EDDI li riduce di 11 volte** (0,15%). I suoi controlli scattano solo nel 2,9% dei casi, ma sono proprio i casi pericolosi: un valore sbagliato viene fermato prima di essere scritto in memoria, passato a una funzione o usato in un salto;
- **CFCSS e RASM da soli aiutano poco** (0,88% e 0,65%): una firma vede solo i bit-flip che cambiano il percorso del programma, non quelli che cambiano un valore;
- **combinando EDDI con una tecnica del flusso resta una sola SDC su 4000** (0,03%).

11.3 Salti condizionali invertiti

Configurazione	masked	rilevati (dati)	rilevati (CFC)	crash	**SDC**	intervallo 95%
baseline	21,2	0,0	0,0	0,0	**78,80**	77,51–80,04
EDDI	4,6	76,8	0,0	0,0	**18,57**	17,40–19,81
CFCSS	7,5	0,0	62,6	0,1	**29,75**	28,35–31,19
RASM	0,0	0,0	99,3	0,0	**0,73**	0,51–1,04
EDDI+CFCSS	2,1	35,5	54,2	0,0	**8,18**	7,37–9,06
EDDI+RASM	0,0	34,0	66,0	0,0	**0,00**	0,00–0,10

Cosa dire:

- **i salti invertiti sono i guasti peggiori**: nei programmi senza protezione il 78,8% produce un risultato sbagliato, perché il risultato dipende da quante volte si ripetono i cicli e da quale ramo si prende;
- **EDDI e CFCSS rilevano solo i salti dei loro stessi controlli.** Abbiamo misurato dai profili quanti dei salti condizionali eseguiti appartengono ai controlli (cioè hanno come destinazione il blocco che chiama il gestore d'errore): il **76,3%** con EDDI e il **62,4%** con CFCSS. Se si inverte uno di questi salti, il programma va nel blocco d'errore e il guasto risulta "rilevato". Infatti le percentuali di rilevamento (76,8% e 62,6%) sono quasi identiche. Quando invece si inverte un salto **del programma**:
  - EDDI non se ne accorge, perché confronta le due copie della condizione *prima* del salto, ma poi esegue un solo salto: entrambe le copie dei dati vanno dalla parte sbagliata e restano coerenti tra loro;
  - CFCSS non se ne accorge, perché il successore sbagliato è comunque un successore lecito (capitolo 4.3);

  e restano il 18,6% e il 29,8% di SDC;
- **RASM rileva il 99,3%**: anche con RASM i salti dei controlli sono la maggioranza (67,4%), ma RASM rileva anche le inversioni dei salti del programma, perché la firma viene aggiornata con la stessa condizione del salto (capitolo 4.4);
- **le 29 SDC rimaste a RASM (0,73%) vengono tutte da un solo salto** di `pid_control`, nella funzione `load_at`:

```
static float load_at(int k) {
    return (k >= 1200 && k < 1300) ? 1.5f : 0.0f;
}
```

Nell'IR questa è una `select` tra due `float`, senza salti: RASM non ha nessun salto da proteggere. Ma il backend x86, senza ottimizzazioni, traduce la `select` con un `jne` (lo si vede con `sdc_sites.py`: i due successori

del salto caricano due valori diversi in `xmm0`). Nessuna tecnica che lavora sull'IR può vedere un salto creato dopo, dal back-end. È un limite interessante da citare all'orale;

- **EDDI+RASM non lascia nessuna SDC**: le due tecniche si coprono a vicenda. RASM vede i salti del programma andati dalla parte sbagliata; EDDI vede il caso di `load_at`, perché duplica anche la `select`: il back-end crea due salti, uno per copia, e se uno dei due sbaglia le due copie del valore differiscono e il controllo successivo se ne accorge.

11.4 Il bug 3 nei numeri

Modello di guasto	SDC prima	SDC dopo	rilevati (CFC) prima	rilevati (CFC) dopo
bit-flip nei registri	0,10%	0,03%	0,5%	1,6%
salti invertiti	**10,53%**	**0,00%**	34,6%	66,0%

Prima della correzione, EDDI+RASM lasciava il 10,5% di SDC sui salti invertiti: peggio di RASM da solo (0,73%), anche se aveva in più tutti i controlli di EDDI. È stato questo controsenso a far partire l'indagine (capitolo 8.4): tutte le 421 SDC erano su salti verso blocchi `VerificationBB` di EDDI dove RASM reimpostava la firma senza controllarla. Dopo la correzione i controlli di RASM rilevano il 66% dei salti invertiti invece del 34,6%, e non resta nessuna SDC.

11.5 Per applicazione

SDC dei bit-flip nei registri (%, 1000 guasti per cella):

Applicazione	baseline	EDDI	CFCSS	RASM	EDDI+CFCSS	EDDI+RASM
aes128	2,2	0,2	0,8	0,5	0,0	0,0
fft	1,3	0,0	0,8	0,9	0,0	0,0
huffman	1,6	0,0	0,4	0,9	0,1	0,1
pid_control	1,7	0,4	1,5	0,3	0,0	0,0

SDC dei salti invertiti (%):

Applicazione	baseline	EDDI	CFCSS	RASM	EDDI+CFCSS	EDDI+RASM
aes128	88,8	22,9	31,8	0,0	10,0	0,0
fft	66,8	13,6	23,7	0,0	5,6	0,0
huffman	89,7	24,9	30,0	0,0	10,7	0,0
pid_control	69,9	12,9	33,5	2,9	6,4	0,0

Le applicazioni si comportano in modo simile. Con 1000 guasti per cella le differenze di pochi decimi di punto (per esempio 0,2% contro 0,4%) **non sono significative**: corrispondono a 2 o 4 casi. Le uniche differenze nette sono quelle già spiegate (le SDC di RASM in `pid_control`).

11.6 Il messaggio in tre frasi

1. La duplicazione dei dati (EDDI) protegge bene dai bit-flip nei registri (11 volte meno SDC) ma costa molto (3,8 volte le istruzioni) e non vede i salti andati dalla parte sbagliata.
2. Il controllo del flusso a firme vede quei salti solo se la firma dipende dalla direzione del salto (RASM sì, CFCSS no); nessuna tecnica sull'IR vede i salti creati dal back-end.
3. Combinare le due famiglie dà la protezione migliore, a patto che le due trasformazioni non si ostacolino: la fault injection ha trovato proprio un caso in cui lo facevano (il bug 3).

Parte III. Pratica ed esame

12. L'integrazione in ASPIS

Il terzo obiettivo è portare le applicazioni nel branch principale di ASPIS come test di regressione. Il lavoro è pronto in `aspis-integration/patches/` come quattro *patch* di git:

Patch	Contenuto
`0001-fix-EDDI-null-safe-...`	bug 1: controlli sicuri con i puntatori nulli (`EDDI.cpp`, `ASPIS.h`)
`0002-fix-EDDI-re-synchronise-...`	bug 2: nuova risincronizzazione dopo le chiamate esterne (`EDDI.cpp`)
`0003-fix-RASM-RACFED-check-...`	bug 3: controllo della firma nei blocchi di EDDI (`RASM.cpp`, `RACFED.cpp`)
`0004-test-add-the-applications-...`	le quattro applicazioni in `testing/tests/c/applications/` e le loro voci in `testing/config/tests.toml`

Cos'è una patch di git. È un file di testo che contiene un commit: autore, data, messaggio e le differenze (*diff*) dei file. Si crea con `git format-patch` e si applica, ricreando lo stesso commit, con `git am`:

```
# come le abbiamo create, dal nostro clone di ASPIS con i 4 commit
git format-patch b66a090..HEAD -o patches/
# come si applicano su un clone pulito di ASPIS
git checkout b66a090
git am patches/*.patch
```

Ogni messaggio di commit spiega il problema e la correzione, così chi deve accettare le modifiche può valutarle una per una. Abbiamo verificato che le patch si applicano senza conflitti su `b66a090` e che il risultato è identico alla versione di ASPIS usata per tutti gli esperimenti.

Come proporle (quando deciderete di farlo). Il modo normale è una *pull request* su GitHub: si fa un *fork* di `HEAPLab/ASPIS`, si applicano le patch su un branch del fork, lo si pubblica e si apre la pull request verso il repository originale, descrivendo il lavoro (si può rimandare al report). Conviene parlarne prima con i supervisori, che sono anche gli sviluppatori di ASPIS: potrebbero preferire ricevere le correzioni separate dai test, o discuterle durante la chiamata. Per ora le patch **non sono state inviate**.

13. Rifare tutto da zero

Requisiti: Linux x86-64 (anche WSL2 o una macchina virtuale), Python 3.11 o successivo con `pytest`, `pytest-timeout` e `matplotlib`, CMake, Ninja, gcc; per rigenerare il PDF del report anche `pandoc` e Node.js con Playwright.

```
# 1. LLVM 21.1.8 già compilato (circa 1 GB)
curl -LO https://github.com/llvm/llvm-project/releases/download/llvmorg-21.1.8/LLVM-21.1.8-Linux-
X64.tar.xz
tar xf LLVM-21.1.8-Linux-X64.tar.xz            # <llvm> = la cartella estratta

# 2. ASPIS con le nostre patch (circa 1 minuto)
git clone --recursive https://github.com/HEAPLab/ASPIS && cd ASPIS
git checkout b66a090
git am <repo>/ASPIS-Project/aspis-integration/patches/*.patch
cmake -B build -G Ninja -DLLVM_DIR=<llvm>/lib/cmake/llvm && cmake --build build

# 3. i test di ASPIS (circa 9 minuti)
#    prima scrivere llvm_bin = "<llvm>/bin" in testing/config/llvm.toml
cd testing && pytest test.py -v > harness_fixed.log && cd ..

# 4. un programma a mano, per vedere cosa succede
./aspis.sh --llvm-bin <llvm>/bin --eddi --rasm <repo>/ASPIS-Project/benchmarks/aes128.c \
    -o aes128 --build-dir /tmp/aes --no-cleanup && ./aes128
less /tmp/aes/out.ll                           # l'IR finale

# 5. la campagna (circa 25 minuti con 4 core)
gcc -O2 -o <dir>/fi <repo>/ASPIS-Project/fault_injection/injector/fi.c
#    adattare i percorsi in fault_injection/campaign.toml, poi:
python3 <repo>/ASPIS-Project/fault_injection/campaign.py <repo>/ASPIS-
Project/fault_injection/campaign.toml
python3 <repo>/ASPIS-Project/fault_injection/export.py <output della campagna> \
    <repo>/ASPIS-Project/results/fault_injection

# 6. tabelle, figure e report
cd <repo>/ASPIS-Project
python3 fault_injection/analyze.py --campaign results/fault_injection \
    --before results/fault_injection/before_fix --out report \
    --harness master=results/compatibility/harness_master.log.gz \
    --harness fixed=results/compatibility/harness_fixed.log.gz
python3 report/build.py
```

Per la parte “prima della correzione del bug 3” bisogna compilare un secondo ASPIS con le sole patch 0001 e 0002 e lanciare la campagna con la sola configurazione EDDI+RASM (le cartelle di output devono essere diverse). Per il log dei test su `master` si lancia `pytest` su un ASPIS senza patch, aggiungendo le nostre voci con `--tests-file config/tests.toml <repo>/ASPIS-Project/aspis-integration/flc_tests.toml`.

Le golden run e gli esperimenti sono deterministici (stessi semi, indirizzi fissi): rifacendo la campagna con gli stessi eseguibili si ottengono gli stessi numeri. Ricompilando le applicazioni, però, l’eseguibile prodotto da EDDI può cambiare un poco, perché EDDI visita le istruzioni in un ordine che dipende dagli indirizzi di memoria del compilatore (usa un `std::set` di puntatori); quindi anche i numeri possono variare di poco.

14. Mappa dei file

Tutti i percorsi sono relativi a `ASPIS-Project/`.

File	Cosa contiene
`README.md`	presentazione del progetto e indice
`benchmarks/aes128.c`, `fft.c`, `huffman.c`, `pid_control.c`	le quattro applicazioni (capitolo 6)
`aspis-integration/README.md`	come applicare le patch
`aspis-integration/patches/0001...0004-*.patch`	le patch per ASPIS (capitolo 12)
`aspis-integration/flc_tests.toml`	le voci dei test aggiunte a `tests.toml`
`aspis-integration/reproducers/null_pointer.c`, `string_literal.c`	i riproduttori dei bug 1 e 2; `handlers.h` contiene i due gestori d'errore
`fault_injection/injector/fi.c`	l'iniettore (capitolo 9)
`fault_injection/campaign.py`, `campaign.toml`	la campagna (capitolo 10)
`fault_injection/export.py`	copia i risultati nel repository e crea `raw.csv.gz`
`fault_injection/analyze.py`	tabelle e figure del report
`fault_injection/sdc_sites.py`	dove sono avvenute le SDC (capitolo 8.4)
`results/README.md`	descrizione dei risultati
`results/compatibility/harness_master.log.gz`, `harness_fixed.log.gz`	i log dei test di ASPIS, prima e dopo le patch
`results/fault_injection/metrics.csv`	costo: dimensione, controlli, istruzioni eseguite
`results/fault_injection/outcomes.csv`	esiti per applicazione, configurazione, modello
`results/fault_injection/raw.csv.gz`	i 48.000 esperimenti, uno per riga
`results/fault_injection/before_fix/`	gli stessi file per EDDI+RASM prima della patch 0003
`report/report.md`	il testo del report (Markdown, con i segnaposto `{{table:...}}` per le tabelle)
`report/build.py`	genera `report.html` e `report.pdf` (pandoc + Chromium) e la versione web
`report/style.css`	lo stile del report
`report/tables/`, `report/figures/`	tabelle e figure generate da `analyze.py`
`report/report.pdf`	**il report da consegnare**

Nella radice del repository: `README.md`, la proposta del progetto (`FLC Project Proposals AY 202627.txt`), le indicazioni dei supervisori (`Buongiorno,.txt`) e, fuori dalla cartella del progetto, questa guida (`guida/`).

15. Cosa è stato tolto (la versione completa)

La prima versione del progetto, prima della riduzione, è nel commit `e1d7e8b` del repository. Per vederla:

```
git show e1d7e8b:ASPIS-Project/report/report.pdf > report-completo.pdf
git checkout e1d7e8b      # tutto il repository com'era (poi: git checkout main)
```

o su GitHub, aprendo il commit `e1d7e8b` e scegliendo "Browse files". Conteneva, in più rispetto alla versione attuale:

- **22 programmi** invece di 4: altre 4 applicazioni (un dizionario su albero binario di ricerca, un piccolo compilatore di espressioni con macchina virtuale, un risolutore LU, un filtro di Sobel) e 14 *microbenchmark*, ognuno dedicato a una caratteristica del C (puntatori a funzione richiamati dalla libreria, funzioni variadiche, struct passate per valore, array a lunghezza variabile, `setjmp` / `longjmp`, campi di bit, …);
- **23 bug corretti** invece di 3, in una serie di 19 commit: tra gli altri, la deduzione dei tipi che non terminava con le struct ricorsive (il compilatore si bloccava), i puntatori a funzione passati a `qsort`, le inizializzazioni delle globali duplicate, le espressioni costanti annidate, `memmove`, le funzioni variadiche, gli array a lunghezza variabile, REDDI, i campi di bit, `setjmp` / `longjmp` nelle tecniche a firme, inter-RASM con le funzioni chiamate dalla libreria. Con quelle correzioni passavano tutte le 1210 combinazioni dei 22 programmi (contro 435 sul branch principale) e 23 combinazioni escluse dalle `black_list` di ASPIS;
- due **strumenti di test** propri, più robusti del sistema di test ufficiale;
- un **iniettore** più veloce, che usava i *breakpoint hardware* del processore (con la chiamata di sistema `perf_event_open`) invece di `int3`, e un terzo modello di guasto (salto a un indirizzo casuale);
- una **campagna di 240.900 esperimenti**: 8 applicazioni, 11 configurazioni (anche sEDDI, FDSC, inter-RASM, RACFED, EDDI+RACFED), 3 modelli di guasto, e un'analisi del rapporto costo/beneficio (*mean work to failure*);
- un report di 19 pagine.

È stato tolto perché richiederebbe a due studenti, a mano, diverse centinaia di ore: molto più di quanto ci si aspetta da un progetto che vale il 30% di un esame da 5 CFU. Le correzioni in più restano comunque utili: se un giorno vorrete proporle ad ASPIS, sono tutte in quel commit (`ASPIS-Project/aspis-integration/patches/`).

16. Domande probabili all'orale

Le risposte sono brevi di proposito: sono il nucleo da cui partire, poi si può approfondire con i capitoli indicati.

1. Cos'è ASPIS e a cosa serve? Un insieme di pass di LLVM che trasforma un programma per renderlo capace di rilevare i guasti transitori dell'hardware, duplicando i calcoli (EDDI e varianti) e controllando il flusso di esecuzione con firme (CFCSS, RASM, RACFED). Funziona sull'IR, quindi vale per qualunque linguaggio e architettura supportati da LLVM. (Cap. 4-5)

2. Perché lavorare sull'IR e non sul sorgente o sull'assembly? L'IR è indipendente dal linguaggio e dalla macchina, ed è facile da trasformare con le API di LLVM. Il prezzo è che non si vede quello che fa il back-end: per esempio i salti con cui traduce una `select` (la SDC residua di RASM in `pid_control`). (Cap. 2, 11.3)

3. Perché ASPIS compila a `-O0` e non ottimizza dopo? Perché un ottimizzatore riconoscerebbe che le due copie calcolano la stessa cosa e le fonderebbe, togliendo la protezione. (Cap. 5.4)

4. Cos'è una SDC e perché è l'esito peggiore? Un risultato sbagliato senza nessun segnale d'errore: chi usa il risultato non sa di non potersi fidare. Crash, hang e rilevamenti sono almeno visibili. (Cap. 3.3)

5. Come funziona EDDI? Dove mette i controlli? Duplica ogni istruzione su copie separate dei dati e confronta le copie prima di store, salti condizionali e chiamate, dove un valore sbagliato uscirebbe dal calcolo. Se differiscono chiama `DataCorruption_Handler`. (Cap. 4.1, 5.6)

6. Che differenza c'è tra EDDI, sEDDI, FDSC e REDDI? sEDDI non controlla prima delle store, FDSC controlla solo in alcuni punti, REDDI protegge solo il codice annotato `to_harden`. Costano meno di EDDI ma rilevano meno o più tardi. (Cap. 4.1)

7. Perché EDDI confronta i valori puntati e non i puntatori? Perché le due copie di un puntatore puntano alle due copie del dato, quindi hanno legittimamente indirizzi diversi. Per sapere che tipo leggere serve la libreria TypeDeductionAnalysis, perché in LLVM i puntatori sono opachi. (Cap. 2.6, 5.6)

8. Come funziona CFCSS e perché non rileva i salti invertiti? Ogni blocco ha una firma; all'ingresso si aggiorna la firma di esecuzione con uno XOR della differenza tra la firma del predecessore e quella del blocco, e la si confronta. Un salto invertito porta in un successore lecito, e l'aggiornamento torna giusto: CFCSS controlla che il percorso sia un arco del grafo, non che sia quello scelto dalla condizione. (Cap. 4.3)

9. E RASM perché li rileva? Prima di uscire da un blocco aggiorna la firma con un valore scelto, con una `select`, dalla stessa condizione del salto. Se il salto va dalla parte opposta, la firma corrisponde all'altro successore e il controllo all'ingresso fallisce. (Cap. 4.4)

10. Quali bug avete trovato? Tre. (1) I controlli di EDDI sui puntatori leggevano anche da puntatori nulli. (2) Dopo le chiamate di libreria EDDI copiava sempre 8 byte dall'originale alla copia, anche dentro le stringhe letterali in memoria di sola lettura: è il crash di tutte e quattro le applicazioni. (3) RASM non controllava la firma nei blocchi `VerificationBB` creati da EDDI, quindi EDDI+RASM perdeva i salti invertiti. (Cap. 8)

11. Come avete trovato il bug 2? Con `gdb`: il crash era sempre una scrittura di 8 byte subito dopo un `printf`, in una stringa letterale (`"kat ct: "`, `"OK"`, `"SUCCESS"`). Poi un programma di 5 righe che lo riproduce, e la lettura di `fixFuncValsPassedByReference` in `EDDI.cpp`. (Cap. 8.3)

12. Come avete trovato il bug 3, visto che nessun test fallisce? Dai numeri: EDDI+RASM aveva più SDC di RASM da solo (10,5% contro 0,73%), il che non ha senso. Con `sdc_sites.py` abbiamo visto che tutte le 421 SDC erano su salti i cui successori cominciavano scrivendo una costante nella firma, cioè blocchi in cui RASM reimpostava la firma senza controllarla. Erano i `VerificationBB` di EDDI, esclusi per errore da un controllo sul nome del blocco. (Cap. 8.4)

13. Come avete verificato che le correzioni non rompano nulla? Con il sistema di test ufficiale di ASPIS: sui 48 test esistenti i risultati con e senza le patch sono identici (2453 superati, 33 falliti, 154 saltati); le nostre applicazioni passano in tutte le 220 combinazioni. (Cap. 7.3)

14. Perché 33 test falliscono anche su `master`? Sono tutti di `cpp_complex`: il programma di riferimento non compila con la libreria C++ della nostra macchina (usa `std::powf`, che libstdc++ 13 non dichiara). Non dipende da ASPIS. (Cap. 7.3)

15. Come funziona il vostro iniettore? Con `ptrace`: una prima esecuzione passo-passo conta quante volte viene eseguita ogni istruzione; poi ogni esperimento sceglie a caso un'esecuzione di un'istruzione (cioè un istante), ferma il programma lì con un breakpoint `int3`, inverte un bit di un registro o la direzione di un salto, lo lascia proseguire e classifica l'esito confrontando l'output con quello di riferimento. (Cap. 9)

16. Perché disattivate l'ASLR? Perché il profilo registra gli indirizzi delle istruzioni: devono essere gli stessi in tutte le esecuzioni. (Cap. 9.1)

17. Perché scegliere l'istante uniformemente tra le istruzioni eseguite? Perché un guasto transitorio può arrivare in qualunque momento: scegliere un'esecuzione a caso di un'istruzione a caso (pesata con il numero di esecuzioni) equivale a scegliere un istante a caso del tempo di esecuzione. (Cap. 9.4)

18. Quanti esperimenti e quanto sono affidabili i numeri? 48.000 (4 applicazioni × 6 configurazioni × 2 modelli × 1000), più 8000 per il bug 3. Ogni percentuale ha un intervallo di confidenza al 95% di Wilson: per esempio la SDC di EDDI è 0,15%, con intervallo 0,07-0,33%. (Cap. 10.3)

19. Perché l'intervallo di Wilson e non quello classico? Perché quello classico funziona male con percentuali vicine a zero (con zero casi dà un intervallo nullo); Wilson resta corretto. (Cap. 10.3)

20. Quanto costano le protezioni? EDDI esegue 3,8 volte le istruzioni e ha il codice 9,2 volte più grande; CFCSS 1,7x, RASM 2,3x; EDDI+RASM 6x. (Cap. 11.1)

21. Perché il codice cresce più dell'esecuzione? Perché ogni funzione esiste in più versioni (`f`, `f_ret`, `f_dup`, `f_ret_dup`), anche quelle non più usate restano nell'eseguibile, ogni controllo aggiunge blocchi, e a `-O0` tutto passa dallo stack. (Cap. 11.1)

22. Perché EDDI "rileva" il 77% dei salti invertiti, se non vede i salti? Perché il 76% dei salti condizionali eseguiti da un programma con EDDI sono i salti dei suoi controlli: invertirne uno manda il programma nel blocco d'errore. I salti del programma invece non li vede. Lo stesso per CFCSS (62%). (Cap. 11.3)

23. Qual è la configurazione migliore? Dipende da cosa si teme e da quanto si può pagare: contro i guasti nei dati serve EDDI, contro i salti sbagliati serve RASM; insieme non lasciano quasi nessuna SDC (1 su 4000 per i registri, 0 su 4000 per i salti), ma costano 6 volte le istruzioni. (Cap. 11.6)

24. Quali sono i limiti del vostro lavoro? Solo registri generali e salti (niente registri in virgola mobile, memoria, guasti multipli); applicazioni piccole; costo misurato in istruzioni e non in tempo; le tecniche sull'IR non vedono i salti creati dal back-end. (Report, sezione 7)

25. Come avete usato l'intelligenza artificiale? I supervisori l'hanno permesso a patto di dichiararlo. Abbiamo usato Claude (Anthropic), tramite Claude Code, per scrivere il codice, eseguire gli esperimenti, cercare e correggere i bug e scrivere il report; noi abbiamo definito obiettivi e perimetro, seguito il lavoro e controllato i risultati. È dichiarato nella sezione 8 del report. Conviene essere pronti a spiegare ogni parte: è lo scopo di questa guida.

26. Le correzioni sono già in ASPIS? No: sono pronte come patch (`aspis-integration/patches/`), verificate sul commit `b66a090`, ma non ancora inviate. Si possono proporre con una pull request, magari dopo averne parlato con i supervisori. (Cap. 12)

17. Glossario

Termine	Significato
ASLR	*Address Space Layout Randomization*: il sistema operativo carica il programma a indirizzi casuali; l'iniettore la disattiva
basic block	sequenza di istruzioni senza salti interne, con un solo ingresso e una sola uscita
back-end	la parte del compilatore che traduce l'IR in codice macchina
CFC	*control-flow checking*, controllo del flusso di esecuzione
CFCSS	tecnica CFC a firme con aggiornamento XOR (Oh et al., 2002)
CFG	*control-flow graph*, grafo dei blocchi e dei salti tra loro
chunk	blocco di 100 esperimenti eseguiti da un processo `fi` con un seme
DUE	*detected unrecoverable error*: errore rilevato (crash, hang, controllo)
EDDI	*Error Detection by Duplicated Instructions* (Oh et al., 2002)
ErrBB	blocco creato da EDDI (o dalle tecniche CFC) che chiama il gestore d'errore
fault injection	iniezione artificiale di guasti per misurare l'effetto delle protezioni
FDSC	*Full Duplication with Selective Checking*, variante di EDDI
front-end	la parte del compilatore che analizza il sorgente e produce l'IR (per il C: `clang`)
golden run	esecuzione senza guasti, il cui output fa da riferimento
hang	il programma non termina entro il tempo limite
harness	il sistema di test di ASPIS (`testing/test.py`)
int3	istruzione x86 di breakpoint (byte `0xcc`): genera un `SIGTRAP`
IR	*intermediate representation*, il linguaggio intermedio di LLVM
jcc	salto condizionale x86 (`je`, `jne`, `jl`, …)
masked	il guasto non ha avuto effetti visibili
opt	il programma di LLVM che esegue i pass sull'IR
pass	trasformazione o analisi dell'IR
patch	file con un commit di git (diff + messaggio), applicabile con `git am`
ptrace	chiamata di sistema con cui un processo controlla un altro (debugger)
RACFED	variante di RASM che aggiorna la firma dopo ogni istruzione
RASM	*Random Additive Signature Monitoring* (Vankeirsbilck et al., 2017)
REDDI	variante di EDDI che protegge solo il codice annotato `to_harden`
regressione	una modifica che rompe qualcosa che prima funzionava
riproduttore	programma minimo che mostra un bug
SDC	*silent data corruption*: risultato sbagliato senza segnalazione
sEDDI	*selective EDDI*: non controlla prima delle store
select	istruzione IR `c ? x : y`, senza salti
SEU	*single event upset*: inversione di un bit causata da una particella
SIHFT	*Software-Implemented Hardware Fault Tolerance*
SSA	*static single assignment*: ogni valore dell'IR è assegnato una volta
TDA	TypeDeductionAnalysis, la libreria che deduce i tipi puntati
VerificationBB	blocco creato da EDDI che contiene i confronti tra le copie
Wilson (intervallo di)	intervallo di confidenza per una proporzione, affidabile anche vicino a 0